

Computer Networks and Internet

104353 — Data Engineering — 1st year
Universitat Autònoma de Barcelona

STUDY GUIDE



By Miguel Hernández-Cabronero
<miguel.hernandez@uab.cat>

UAB



This document was compiled on May 28, 2026.

License

Copyright 2024-* © Miguel Hernández-Cabronero <miguel.hernandez@uab.cat>.

This document and the accompanying materials are **free to distribute and modify** for non-commercial uses, provided you cite its author(s) and maintain the same sharing terms as the originals (see below).

The latest version of this document and the accompanying materials can be obtained from <https://github.com/miguelinux314/uab-xoi>.



Licensed under the **Creative Commons Attribution-NonCommercial-ShareAlike 4.0 License** (the "License"). You may not use this file except in compliance with the License. You may obtain a copy of the License at <https://creativecommons.org/licenses/by-nc-sa/4.0/>. Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

Credits

The structure and contents of this guide are heavily based on the class materials produced by Sergi Robles and other professors of the DEIC department of UAB. Many of the visual contents were created by third-party artists and released under compatible licenses:

- [Mathias Legrand and Vel](#): base $\text{\LaTeX}$ template.
- [TheDigitalArtist/pixabay](#): Front cover and Chapter 6 header.
- [victorsteeppixabay](#): Chapter 1 header.
- [OpenClipart-Vectors/pixabay](#): Chapter 2 header.
- [D-Kuru](#), [Niridya](#), [Joe Ravi](#), [Shaddack](#) / Wikimedia Commons (WC) : Ex. 2.6
- [Timwether](#), [Timewalk](#) / WC : Ex. 2.7.
- [deepai](#) : Iceberg Section 2.2.
- [Chetvorno](#) : Antenna diagram Section 2.2.
- [Berserkerus](#) : Modulation diagram Section 2.2.
- [Graham Rhind](#) : Chapter 3 header.
- [mrcolo](#) : Chapter 4 header.
- [Geek2003](#) : Switch in Section 3.1.
- [ignartonosbg](#) : Chapter 5 header.
- [Toffelginkgo](#) : Chapter 7 header.
- [Doodles43](#) : Chapter 8 header.
- [Bru-no](#) : Router ports in Section 8.
- [PublicDomainPictures](#) : Chapter 9 header.
- [joo213](#) : Chapter 10 header.
- [Frank_Rietsch](#) : Mission intro header.
- [succo](#) : Mission 1 icon (chapter 11.1).
- [Pixabay](#) : Mission 2 icon (chapter 11.2).
- [mmisof](#) : Mission 4 icon (chapter 11.4).
- [DeepAI](#) : Mission 5 icon (chapter 11.5).
- [1zoom.me](#) : Mission 6 icon (chapter 11.6).
- [DeepAI](#) : Mission 7 icon (chapter 11.7).
- [StockSnap](#) : Mission 9 icon (chapter 11.9).
- [jarmoluk](#) : Index of Concepts header.

Contents

1	Intro: Computer Networks and Internet (XOI)	5
----------	--	----------

I

Computer Networks

2	Piercing the analog-digital veil	8
2.1	Information ↔ Data ↔ Binary messages	8
2.2	Analog ↔ Digital	10
3	Where are you?	12
3.1	Please contact me	12
3.2	I need to find you	13
3.3	How do I get there?	14
4	Is this yours?	16
4.1	I have a delivery for you	16
4.2	Please fill in the form	17
4.3	They keep coming	18
5	A Stack of Layers	20
5.1	Sending data	20
5.2	Receiving data	21

II

Internet

6	The Internet	25
6.1	The TCP/IP stack	25
6.2	TCP/IP protocol overview	26
6.3	Who is the Internet boss?	26
7	Layer 2: Network (LAN) communication	28
7.1	Layer 2 addressing	29
7.2	Ethernet Layer 2 protocol	29
8	Layer 3: Internet(work) communication	32
8.1	Layer 3 addressing	33
8.2	IP – Internet Protocol	36
8.3	IP routing	37
8.4	IP subnetting	39
8.5	IP fragmentation	41

8.6	ARP – Address Resolution Protocol	42
8.7	ICMP – Internet Control Message Protocol	44
9	Layer 4: Transport	45
9.1	Layer 4 addressing	46
9.2	UDP – User Datagram Protocol	46
9.3	TCP – Transport Control Protocol	47
9.4	TCP: “Connecting”	49
9.5	TCP: “Connection established”	50
9.6	TCP: “Closing connection”	52
10	Layer 7: Application	56
10.1	DNS – Domain Name System	56
10.2	DHCP – Dynamic Host Configuration Protocol	58
10.3	Autonomous Systems	60

III**Missions**

11	Missions	63
11.1	Mission 1: Sharing is caring	64
11.2	Mission 2: Infinite scroll	65
11.3	Mission 3: Herding chaos	66
11.4	Mission 4: Inter { planetary net	67
11.5	Mission 5: Money is the root	68
11.6	Mission 6: Annoy two whole planets	69
11.7	Mission 7: To TCP or not to TCP	70
11.8	Mission 8: You talking to me?	71
11.9	Mission 9: Interplanetary Hallway	72

1. Intro: Computer Networks and Internet (XOI)

What you (and out taxes) paid for

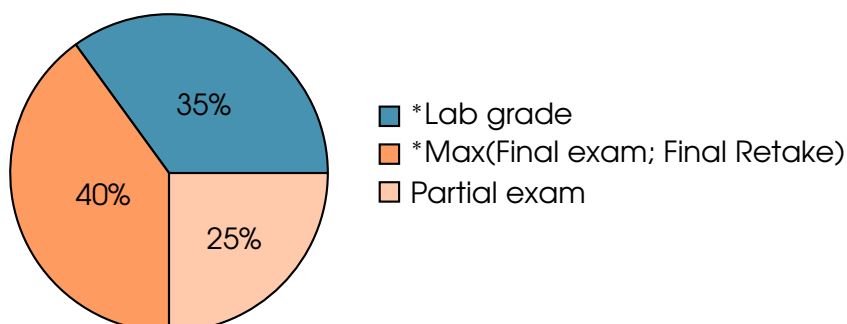
	Monday	Tuesday	Wednesday	Thursday	Friday
8:00					
8:30					
9:00					Labs PLAB/811
9:30					
10:00					Labs PLAB/812
10:30					
11:00				Problems PAUL/811	Labs PLAB/813
11:30					
12:00				Problems PAUL/812	Labs PLAB/814
12:30					
13:00		Lecture Full class			
13:30					
14:00					
14:30					
15:00					

- 13× 2h **Lecture** sessions
- 12× 1h **Problem** sessions
- 12× 1h **Lab** sessions
- Unlimited **Office hour** sessions (*tutories*),
individual or group: send email
- 90h **Autonomous work** (yours)
- 1× **Partial exam** + review
- 1× **Final exam** + review
- 1× **Final exam retake** + review
- 3× **Lab submissions** feedback
- 1× **Lab exam** + review

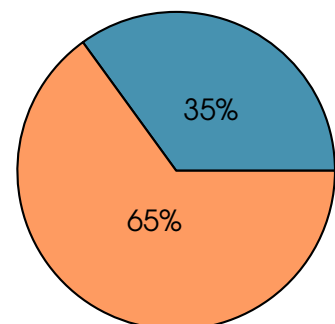
Evaluation

The best of the two following options will be used to determine your grade:

Option A



Option B



*: A passing grade ($\geq 50\%$) in the part is required to pass the subject.

This Guide

This guide is designed to help you progress through all parts of the “Computer Networks and Internet” (XOI) course.

Here you can find written materials, exercises and other tools to make the most of your effort and help you acquire valuable skills. It is strongly recommended that you become familiar with this guide and use it all throughout the semester, starting the very first week of class:

- During the lectures, we will use it to present and refer to new and previously visited concepts. We will also use it to drive activities and problem sessions.
- At home, it can help you gather and organize new knowledge, as well as to find useful exercises to practice your newly acquired skills.
- After you complete the course, it can be useful as an index of contents and pointers to useful reference materials.



This guide is not a book, a reference material or even a complete set of class notes, and **it is not intended to substitute your own notes**. The guide’s goal is rather about presenting you with new, interesting questions than about providing complete answers. This guide does not substitute continuous, active class attendance and autonomous work at home. Instead, you are encouraged to use the guide to help you understand the lectures, and vice-versa.



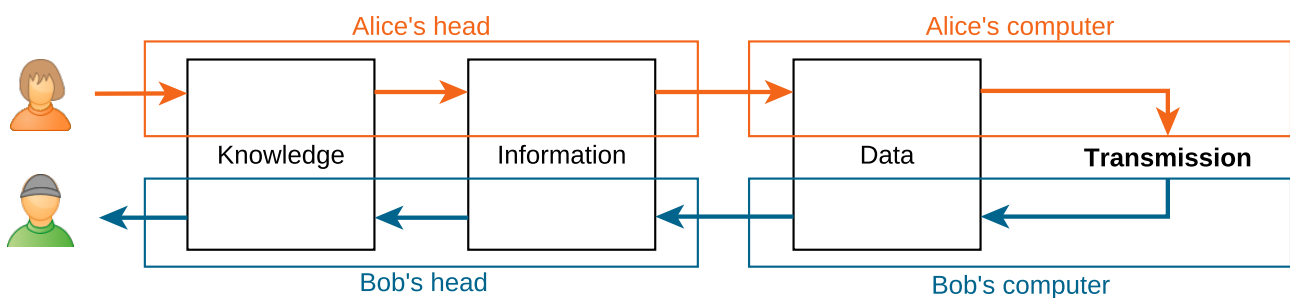
Computer Networks

How do computers exchange data? How can we locate and communicate with other machines located around the world? This part of the guide explores some of the main challenges we need to face if we want to interconnect devices at the planetary scale, and beyond.

2	Piercing the analog-digital veil	8
2.1	Information ↔ Data ↔ Binary messages	
2.2	Analog ↔ Digital	
3	Where are you?	12
3.1	Please contact me	
3.2	I need to find you	
3.3	How do I get there?	
4	Is this yours?	16
4.1	I have a delivery for you	
4.2	Please fill in the form	
4.3	They keep coming	
5	A Stack of Layers	20
5.1	Sending data	
5.2	Receiving data	

2. Piercing the analog-digital veil

We send and receive **messages** constantly in our personal, professional and political spheres. The almost-instantaneous availability of virtually any desired **information** is a key characteristic of the current era. When we use a computer of any kind to share or retrieve knowledge, first it must be transformed into **data** that can be understood both by humans and computers. This can be imagined as a sequence of steps:



Computers work with and transmit digital **data**, *i.e.*, sequences of **binary 0 and 1 symbols**. However, data **transmission** happens in the real world, where 0 and 1 are ideas and not tangible objects.

This chapter studies how those digital symbols *pierce the veil* from the digital to the analog (physical) world on transmission, and back from analog to digital on reception.

2.1 Information ↔ Data ↔ Binary messages

Knowledge, **information**, and **data** are not the same thing, although they are often used interchangeably. In technical contexts, they should be used and interpreted accurately.

Exercise 2.1 Suppose we are outdoors and we want to tell our friend about the weather. We have knowledge about everything related to the weather, because we are there. For our communication, we need to focus on part of that knowledge, and decide what information to send. For instance, we could want to communicate the temperature, and say something like “the temperature is about 22°C”. There are many ways in which we can store that information as data inside a computer. Most likely, we will represent the number 22 as part of those data. How would you define knowledge, information and data?

Even though there are many types of data, *i.e.* numerical, textual, visual, scientific, *etc.*, these are eventually represented by numbers. For instance, one could use **ASCII encoding** to represent the string “Bobby” as the following sequence: 66,111,98,98,121.

Inside a computer, those numbers are stored in binary registers, *e.g.* in the CPU and in the RAM. However, there are multiple ways of representing numerical data in binary format. More specifically, we need to pay close attention to at least the following aspects whenever reading or writing data:

- **integer** or with decimals?
- **bitdepth**? (*e.g.*, 8 bits per number)
- **signed** or **unsigned**?
- For multibyte numbers: **big endian** or **little endian**?
- For numbers with decimals: **floating-point** or **fixed-point**?

Exercise 2.2 Explain **how to complete the second row given the first** (and vice-versa), and what assumptions you need to make in each case.

Number	7	2	-3	0.75	260
Binary	00111	00000010	1101	1100	0000 0100 0000 0001

How about $\tilde{a} \leftrightarrow 1100\ 0011\ 1010\ 0000$?

Sometimes we need to inspect the contents of a binary register (e.g., for debugging or other analysis purposes) or to set those contents manually. For humans, it is inconvenient and very prone to error to handle binary strings longer than a few bits. When we need to inspect or modify the contents of a binary register (e.g., for debugging purposes), we often use base 16, i.e., **hexadecimal** notation.

In hexadecimal, there are exactly $16 = 2^4$ different digits (0 to 9, a to f) with decimal values from 0 to 15, each of which represents exactly 4 bits. When we writing hexadecimal values to a computer, i.e., in source code, hexadecimal values are typically preceded by **0x**, e.g., **0x58a5b0**. Similarly, binary expressions are preceded by **0b**, e.g., **0b0011**.



- Spaces and even line breaks between digits do not carry any meaning, they are only used to facilitate visual inspection.
- In some texts, when multiple bases are applicable in a context, they are shown as subscripts as in **58a5b0₁₆** and **0011₂**.

Exercise 2.3 Consider the following message, which is composed of a **concatenation** of 3-bit unsigned integers: **a4 3f 20**.

- How many bits and bytes long is it?
- **What are the first five integers** contained in the message?

Most often, we will let our code handle data manipulation. To do that, we need **bitwise** and **boolean logic** operations such as the following

Operation	bitwise AND	logic AND	bitwise OR	logic OR	left shift	right shift
Python	&	and		or	<<	>>

Other useful python tools are the `bin()` and `hex()` functions, that convert an integer into its binary and hexadecimal representation, respectively, and `int()`, which can parse a string describing a number and return that number. We can also control the format in which we show numbers, e.g., `print(f'{n:08b}')` would print the value of variable `n` in binary form, using at least 8 positions and filling the empty leading positions with zeros (more in the [python docs](#)).

Exercise 2.4 Consider the code shown next. The numbers printed when run are of special importance in networking and computer science. What do they have in common? (Hint: You may want to modify the code to show their binary representations).

```

.py .out snippets/bitwisemanipulation.py
1 a = 0xff
2 print(a)
3 for _ in range(8):
4     a = (a << 1) & 0xff
5     print(a)

```



For code listings like the one above, you can download the source code and its output.

2.2 Analog ↔ Digital

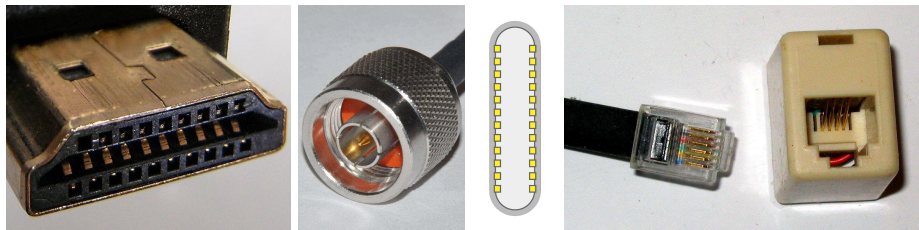
Once we decide what bits to transmit, we need to find a way of making those bits available to another machine at a distance. First, we need to decide what **medium** to use and what physical properties we can control and monitor. Popular choices include using voltage on **copper wires**, sending light over **optical fiber** and manipulating the electromagnetic **field** using **antennas**.

Exercise 2.5 Ponder:

- Can we make a copper cable as long as we desire?
- What medium do Wi-Fi connection use?
- Do we need optical fibers to perform light-based communication?

If we opt for copper wires, we can put several in **parallel** and send more data at the same **clock** speed. This, however, comes at the price of additional complexity and cost than **serial** designs. Voltage interferences in the **data lines** may happen for a number of reasons, including physical phenomena such as electromagnetic induction in the wire. Notwithstanding, cables compliant with modern data transmission standards (e.g., IEEE 802.3) often employ **twisted pairs of cables** and shielding among other strategies to guarantee bit errors below 1 in 10^{12} bits.

Exercise 2.6 Consider and identify the following wire connectors:

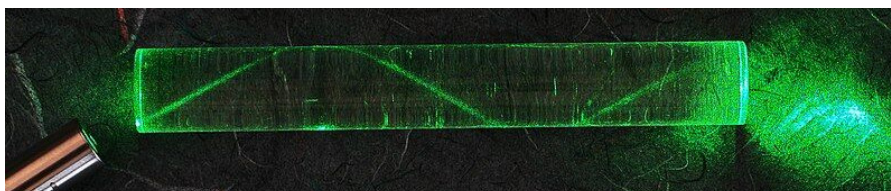


Have you ever been diving in a swimming pool, looked up and the water was mirror-like? Optical fiber cables work of this phenomenon, called **refraction**. Light that enters the fiber through one end stays inside until it exits through the other end. The coating of the fiber is important to support its integrity, but that coating does not participate in the “bouncing” of light when advancing through the fiber.

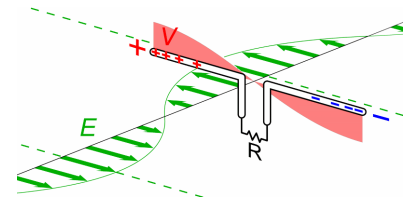
In a **single-mode** optical fiber, the sensor at the receiving end detects the presence or absence of a single wavelength range. These cables are relatively thin and cheap, but carry a single signal. Another option is to transmit multiple wavelengths (e.g., using several laser sources) at the same time, through the same fiber. At the receiving end, a prism is used to divide the beam of light back into its **monochrome** components, which are sensed separately. In this way, **multi-modal**

optical fiber allows **multiplexing** several signals, greatly increasing the effective transmission rate.

Exercise 2.7 When a beam of light enters an optical fiber, it does so with a certain angle of attack. This angle affects the time it takes to reach the other end. If we project a cone of light, light enters the fiber at multiple angles of attack. What happens at the other end if we **turn this cone of light on and off** very quickly? Is it relevant whether we use monomodal or multimodal beams?



Even though there are many classes of **antennas**, their main purpose is to detect electromagnetic fields and/or to create them. The Maxwell-Faraday equation tells us that *changes* in the electric **field** $\vec{E}$ create a perpendicular magnetic **field** $\vec{B}$, and the other way around.



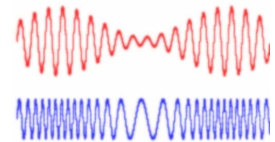
A **dipole** antenna like the one to the right uses this concept by applying an alternating voltage V , which generates electromagnetic waves that propagate outwards the dipole axis.



When we emit some energy E from a point in all directions, that energy is distributed around a growing sphere. Since the area of a sphere of radius r is $4\pi r^2$, the amount of energy that reaches a point at distance r will be in the **ballpark** of E/r^2 .

This is sometimes referred to as the inverse square law, and is the reason why currents induced in the receiver's antenna are usually between nanovolts (10^{-9} V) and picovolts (10^{-12} V). Surprisingly, this is enough for modern detectors to read the data signal.

Regardless of the method chosen to let the data travel, the veil between analog signals and digital data must still be pierced. One way of doing this is **modulation**: a base sinusoidal signal called **carrier** is produced, and the data are encoded by modifying this carrier. For instance, one can change the **amplitude** of the carrier (ASK), its **frequency** (FSK), or even the amplitude and the **phase** at the same time (QAM).



Exercise 2.8 The figure above displays an example of amplitude modulation and of frequency modulation. Which is which? For each case: is that **modulation transmitting an analog or a digital signal**?

Exercise 2.9 What's the **difference between bandwidth and transmission speed**? Why do you think they are interchangeably used in non-technical contexts?

3. Where are you?

Connecting two computers makes them much more useful than two isolated machines. However, their true potential is only unlocked when they can be connected to *lots* of other computers.

Practical limitations including cost and geographic location make it impossible to directly connect every pair of computers. Instead, devices are distributed across a graph of interconnected networks, which may extend even beyond the scale of a planet.

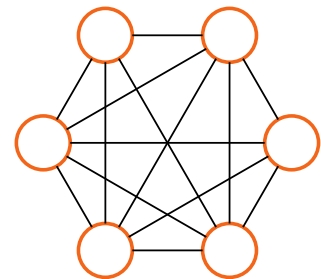
Whether locally within physical reach or in another continent, an addressing system is needed to identify, locate and route messages so that they reach their intended peers, just like we do with physical locations.

3.1 Please contact me

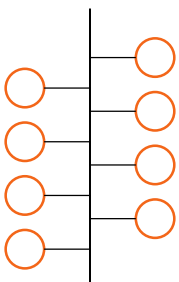
When only 2 **devices** are involved, one may use a **point-to-point** connection. In this type of connection, there is only “this side” and “the other side” of the cable. Moreover, when a device wants to communicate with the other end, there is only one link (e.g., one cable) to choose from, so there is no possible confusion. Since we can operate this connection with just 2 **network cards** and 1 cable, point-to-point connections are viable for $N = 2$ devices.

If we want to connect $N > 2$ devices, a naïve option is to connect every device with each other in a **complete graph**. Now every device needs to handle $N - 1$ cables to the other devices, but communicating with any other device is as easy as choosing the correct cable and establishing a point-to-point connection.

The main issue with this approach is that the total number of connections is $N \cdot (N - 1) / 2 = O(N^2)$ (more on that in your trusted *Discrete Math* course). This means that, in order to have $N = 10$ devices connected in this way, you would need for instance 40 cables and 90 **network cards**. For $N = 1000$ devices, there would be half a million point-to-point connections, which is already quite unmanageable.



$O(N^2)$ is notation for a **quadratic** complexity bound. This is *not* the same as **exponential** and should never be confused. ([read more...](#))



There is one trick up our sleeves: **data buses**, where one or more data lines that are simultaneously connected to multiple devices. This retains the simplicity of point-to-point connections, because each device needs to handle only 1 cable and can use it to communicate with all other devices. The cost-effectiveness is also retained, since only N network cards for N devices ($O(N)$).

The main advantage of data buses is also their main weakness: all devices send and receive data using the same data line, but they cannot do it at the same time because there is only one line. If two or more do, there is a **collision** that makes data transmission impossible while it lasts.



As computer networking was developed, people experimented with many alternatives to the bus **topology**. One curious example is **Token Ring**, where devices are

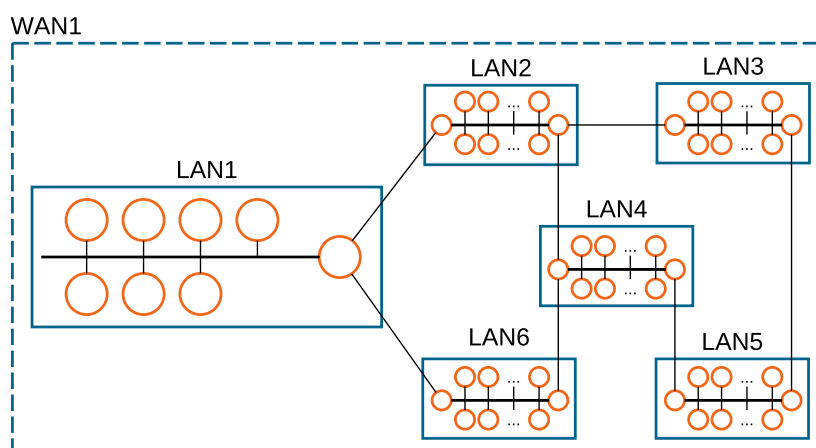
connected around a circle, and messages are passed around in a single direction.

In practice, it is not trivial to implement a bus technology where new devices can be added over time. Historically, this was sometimes done by physically piercing the main bus cable, and adding a new cable towards the new computer being connected. Later on, this was simplified with devices called **hubs**. They worked similarly, but they offered pre-built **ports** and removable connectors. Nowadays, hubs have been replaced by **switches**. Switches offer the functionality of a bus, connecting every device with each other, and also prevent collisions. For obvious reasons, structuring connections around a switch is called a **star topology**.

Exercise 3.1 What is **needed in a switch** that is not needed in a hub?



Bus topologies work great at local scales, and are extensively employed in **local area networks (LANs)**. However, it is not cost-effective, (sometimes not even physically possible) when devices are far apart. Instead, devices can be organized in separate LANs, each one conceptually a bus, and then these separate LANs can be connected using cheaper, feasible point-to-point connections. Considered together, these devices would then form a **wide-area network (WAN)**.



3.2 I need to find you

Once again, the main advantage of data **buses** is also their main weakness: all devices send and receive data using the same data line. Even if collisions don't happen, every message sent into the bus is received by all devices. However, what we actually want is for our message to reach its destination, and only its destination. Let's call this the *find-my-device problem*.

The find-my-device problem is not unique of buses or **LANs**. On the contrary, it becomes very important when we consider **WANs**, where not all devices are directly interconnected. In this case, we need to make sure we can **route** our messages between different LANs, and that they reach their destination.

Surprisingly, not even point-to-point connections are safe from the find-my-device problem. Assuming devices A and B are connected that way, two different programs may want to exchange information at the same time, e.g., some alarm control software and a music streaming app. In this scenario, you want the **client** and **server** of the alarm monitoring **service** to communicate with each other but not with the music streaming service, and vice-versa.

The most common solution to this problem is to use identifiers that let us differentiate between devices or even between running **processes** inside an **operating system (OS)**. Some of these identifiers are referred to as **addresses**, precisely because they are used to find and reach a

remote device.

Addresses are typically *numerical* identifiers, that is, just a number. As such, they can be expressed in different bases, including decimal and **hexadecimal**. These numbers are drawn from a predefined set, and the choice of that set determines the maximum number of elements we can uniquely identify.

🎵 Addresses expressed as **d8:43:ae:61:ed:f1** or **192.168.1.1** (as it will be seen later in the course) are also numbers we could have expressed as **237785200061937** and **3232235777**, respectively (which is not as convenient).

Exercise 3.2 How many elements can be uniquely identified (at most) with an address we express using 5 bits? How about 10? How about 20? Express the general solution mathematically.

Exercise 3.3 Suppose there are 1000 machines in a LAN. How many address bits are needed? How about 800 machines? And 1100? Express the general solution mathematically.

🎵 In 2019, we run out of Internet (v4) addresses. Whoops!

3.3 How do I get there?

In many scenarios, multiple addresses are needed to perform the desired **end-to-end** communication. For instance, we may need to identify a single device within a **LAN**, and that LAN within a **WAN**. Sometimes, we will need to identify a **process** (a running program) within that device. This is not unlike someone paying you a visit in person:

Joseph Edgar Foreman
123 South Fake St.
Adams County
Ohio

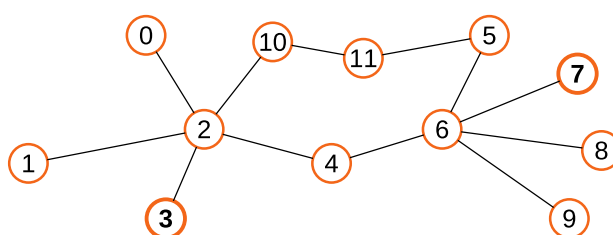
Process #1717403
Device #51688798
LAN 3141 (Data Engineering Labs)
WAN 442 (UAB)

Whatever addressing system we use, it must contain enough information to locate and reach the destination, *i.e.*, to **route** our messages through the graph of connections. The process of routing involves multiple steps or “jumps” across networks until the destination LAN is reached.

The name **router** refers to a type of network device whose job is to forward these messages across networks. These devices must contain enough information so that, when handed a message with a destination **address**, they can decide what **network interface** to use. In turn, non-router devices only need to be configured so that they can reach the next router.

🎵 At home, your **router** typically also plays the role of a **switch**, but those are different concepts.

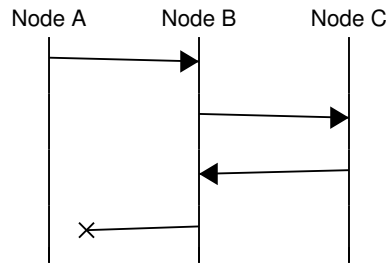
Exercise 3.4 This figure represents a handful of devices and their physical connections. These devices can communicate only by sending messages through those connections. Notwithstanding, ③ wants to send a message to ⑦.



- Can you identify any devices likely to be acting as a **switch**? And as a **router**?

- What's the **minimum number of messages** sent so that ③ can contact ⑦, and ⑦ can reply to ③? List them in the order they are produced.
- What information needs to be included in those messages so that the communication ③ ↔ ⑦ may take place?

Swimlane diagrams like the one below are used to represent the interactions (e.g., message exchange) between actors (e.g., network devices). In them, the vertical axis represents time, which is invaluable to identify cause-effect relations.



Exercise 3.5 Based on your answer to Ex.3.4, **produce a swimlane diagram** that represents the message exchanges. For each message (e.g., above the arrows), include the addressing information present in it.



4. Is this yours?

Information travels through this graph of networks in the form of packets, which contain a small amount of information and meta-information.

Packet format must be agreed upon by both ends of the communication, so that those packets can be efficiently and automatically produced and interpreted.

When packets travel through one or more networks, they can get lost or reordered. If a continuous stream of data must flow between two computers, packets must contain mechanisms to detect losses and restore the proper order.

4.1 I have a delivery for you

It is not feasible to establish physical connections between each pair of computers that want to communicate. Instead, relatively few connections are shared to carry messages between many pairs of devices.

If a single device transmits data continuously for a long time, that connection is blocked and may become a bottleneck that prevents other devices to communicate. To avoid this problem, connections limit the maximum amount of data that can be sent without interruption. As a result, devices are forced to send data in discrete bursts called **packets**. The exchange of these type of messages across one or more networks is called **packet switching**.



Leonard Kleinrock, one of the pioneers of internetworking, disputes the authorship of packet switching, which is often attributed to Paul Baran and Donald Davies.

Packets need to be small so that connections are not blocked for too long. At the same time, we want packets to contain as much data as possible, because each packets contain **overhead** data (such as the addresses discussed in Section 3) that need to be sent in addition to the user's payload data. Networks set the maximum packet length using a parameter called **Maximum Transmission Unit** (MTU), e.g., 1500 bytes in Ethernet.

Packet transmission across a **data link** must be done carefully, particularly when that link is a **bus** shared by multiple devices. Each packet must be individually distinguishable from the rest, which can be done via at least three strategies:

1. *Fixed length*: the length of all packets is the same. The sender and the receiver must have agreed to this value in advance.
2. *Explicit length*: the length of the packet is included in the packet, e.g., using the first byte of the packet. The sender and the receiver must agree on how this information is included and how to interpret it.
3. *Escape sequences*: Certain binary **escape sequences** within the data have a special meaning. For instance, one could define the byte **11110000 (0xf0)** to mean "end of packet": these bits must appear after each packet, and only then. Then the sender could start transmitting the contents of a packet, followed by **0xf0**. Both parties must agree on what escape sequences to use, what they mean, and how to send data equal to those sequences (e.g., it should be possible to send **0xf0f0f0** without triggering an "end of packet" until all bytes are transmitted).



It is also possible to signal the *beginning* of a packet. In that case, a preceding sequence called **preamble** is used.

Exercise 4.1 All strategies described above are used nowadays in real scenarios, depending on the features, costs and trade-offs of each one.

Discuss which of these strategies would be most appropriate for each of these scenarios:

- A single manufacturer designs the hardware and software that communicates two endpoints. A single **data line** must be **multiplexed** for multiple control commands as well as multiple data streams. The priority is efficient power and buffer usage.
- Multiple devices share a **bus**. They occasionally send messages of different lengths, but not a huge volume of data. The priority is cost.
- Multiple devices share a **bus**. They continuously send messages of different lengths, trying to maximize the effective transmission rate through the **channel**. The priority is **throughput**.

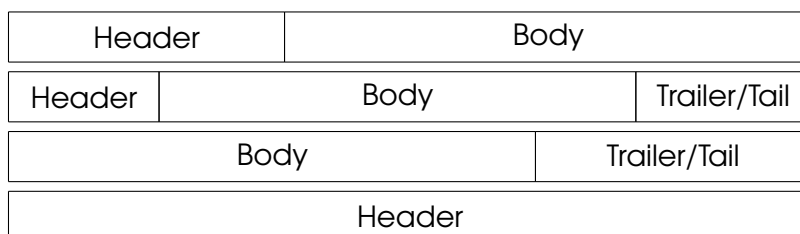
4.2 Please fill in the form

Regardless of the strategy employed, data packets typically comprise two parts:

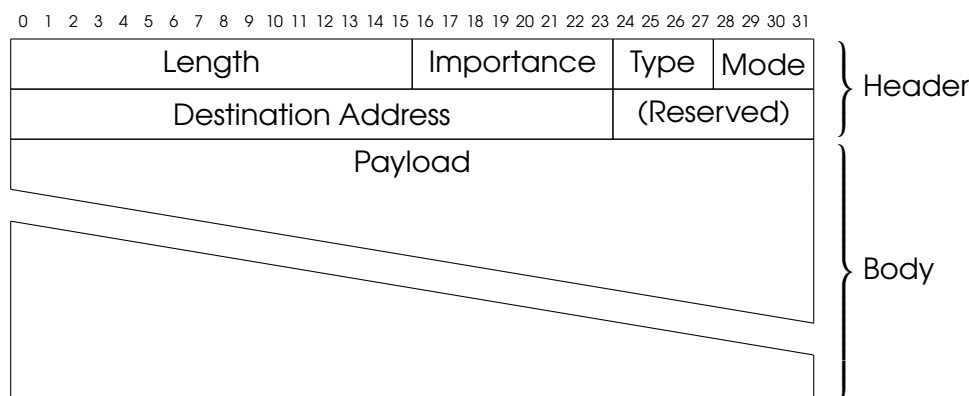
- Data **payload**: the information that needs to be transmitted, and
- **Metadata**: the meta-information needed to deliver the payload.

Length might be one of the metadata **fields** included in the packet, along with others that help identify their type and function. The location of payload and metadata, as well as the exact metadata fields included, their length and their meaning must be known by both ends of the communication. All these decisions constitute the **packet format** or **packet structure**.

One of the most common ways of arranging payload and metadata is with a **header-body** format. In this case, all meta-information is included first, followed by the data. Other formats may include a **trailer** (also known as **tail**, most often used for **error detection** and correction), in combination with the header, or replacing it. All of the following configurations are possible.



The format of the payload in a packet is normally user-defined. The format of the header, however, is strictly defined so that it can be easily produced and **parsed**. This format is often presented using a diagram that helps identify the individual bit and byte positions, like in the following (not-real) example:



Fields may also have fixed or variable lengths, which do not necessarily align with **byte boundaries**. Fields may also have length equal to 1 bit, in which case it is normally called a **flag** or flag bit.



In diagrams like the one above, multiple lines (rows) are often used so that they can be easily printed and read. In that case, the meaning is the same as if all fields had been presented along the same line (row).

Exercise 4.2 The following packet contents were **sniffed** out of a **network**, expressed in **hexadecimal**. Assuming the packet has the format of the example above:

- Indicate the value of each field (length, importance, type, mode, destination address, and payload).
- Can you guess the meaning of the payload?

00 0c 00 f3 bb bb bb 00 68 65 6c 70

Exercise 4.3 The following code accepts two arguments: (1, sys.argv[1]) the **path** of an input file, and (2, sys.argv[2]) the **path** of an output file. The first 255 bytes of the contents of the input file are read, they are formatted as a packet, and the bytes of that packet are output to the output file.

```
1  with (open(__file__, "rb") as input_file,
2      open("/dev/stdout", "wb") as output_file):
3      # Read at most 255 bytes from the file
4      payload: bytes = input_file.read(255)
5      # The + operation concatenates bytes
6      output_file.write(bytes(len(payload)) + payload)
```

- Describe the **packet format** used in the code, and its limitations.
- Extend the packet format so that
 - Packets can be longer than 255 bytes
 - The **address** of the destination device can be encoded
- Provide a byte diagram of the new format, as well as an explanation of the addressing system you are using.
- Modify the code so that it outputs packets of your proposed format.



Languages like Python can make a distinction between files containing text and files containing binary data. In the code above, files are open in **binary mode** using modes **'rb'** and **'wb'**.

When open in binary mode, file bytes are directly represented by a **bytes** object, an array of integers between 0 and 255. In **text mode**, those bytes are processed (**decoded**) further by the **open** method, and returns a string that might contain special characters like accents or non-latin glyphs. ([read more...](#))

4.3 They keep coming

When developing applications that use networking capabilities, it is often useful to imagine communication as a **stream**, *i.e.*, as a conceptual pipe where we put our data (any amount of data) in one end, and it comes out at the other end. If we have this capability, then it becomes much easier to send files of any size, and to transmit a never-ending amount of audio or video. Unfortunately, all we have to simulate those streams are packets.

Packets often need to be forwarded through multiple networks. **Routers** are not perfect and are sometimes inoperative or improperly configured, so not all packets arrive to their destination. Moreover, different packets may be delivered through different routes that may be of different length, so packets may arrive out of order.

If we want to simulate a **stream** of data, packets must contain enough meta-information in them so that losses can be detected and the correct order can be restored. This introduces an **overhead** that is not suitable in all scenarios –e.g., very low latency–, so some applications base their network communication in **messages** instead of streams.

When streams are required, the concept of **offset** is used to reassemble multiple messages into a single stream. Each packet contains some bytes of the data stream, and the offset indicates the exact location in that stream.

Exercise 4.4 The following diagram describes a stream of 48 bytes produced by a source node, as well as the packets it was split into before transmission:

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47

<u>Packet 0</u> Offset 0 Length 10	<u>Packet 1</u> Offset 10 Length 14	<u>Packet 2</u> Offset 24 Length 6	<u>Packet 3</u> Offset 30 Length 18
--	---	--	---

- Describe the minimum header that can be used to transmit this stream.
- Assuming that the recipient receives Packet 2 first, followed by Packet 0 and other packets are lost: what bytes of the stream are known by the destination node, and what bytes are unknown?
- How can the source node know that some packets were not delivered?
- What would happen if a bogus message is received by the destination, with offset 7 bytes and length 4 bytes?

5. A Stack of Layers

Modern computer communication is achieved using a **stack** of network **layers**. Each stack focuses on some problems and is responsible for providing certain features, e.g., **addressing** and correct **sequencing** (introduced in the previous chapters).

Each **layer** defines a **send/receive function pair**, which is responsible for some of these features. Each layer uses the **send** and **receive** of the layer below, i.e., the features of one layer rely on the features of all layers below it.

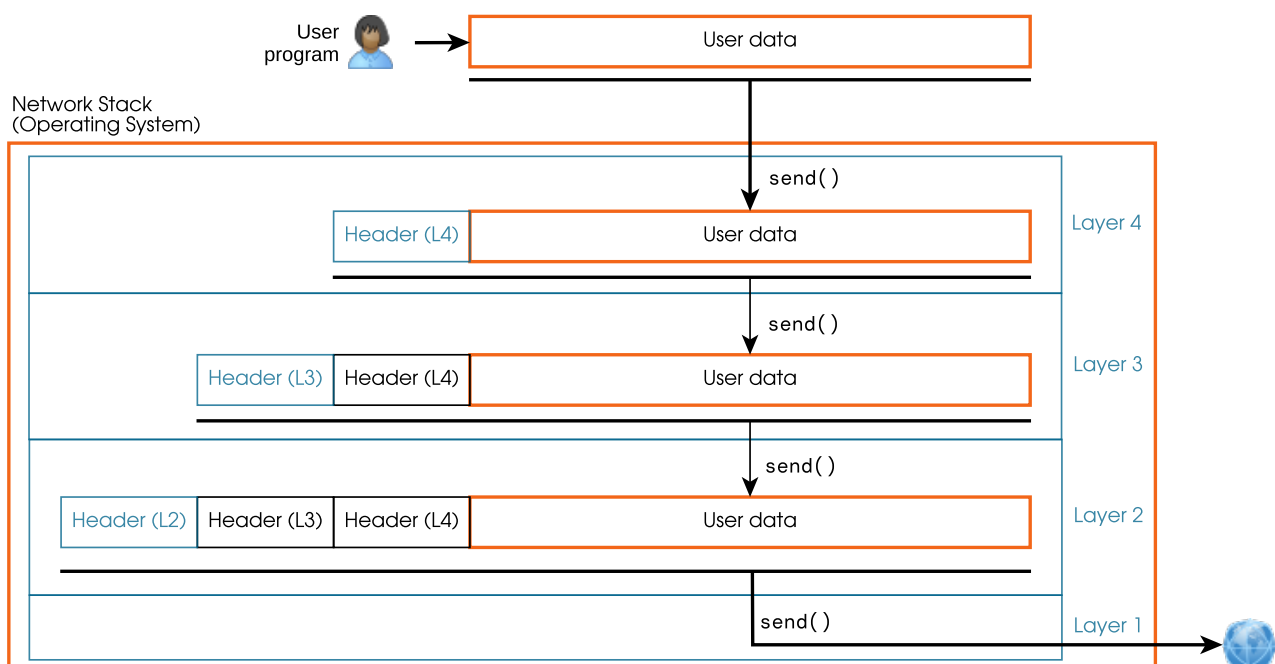
To coordinate layers and enable **virtual** communication between them, each layer defines a different **packet format**, which **encapsulates** a packet of the layer above it.

5.1 Sending data

To implement these capabilities, each layer transmits not only the data provided by the layer above, but also a header with all the meta-information required for that capability. Thus, each layer needs to define the **packet format**, adding some **overhead** (typically a **header**) to the **payload**, e.g.:



When a user program wants to send some data, it uses the **send** method of the *top* layer of the stack, which eventually results in some data being transmitted by the physical layer, e.g., through a cable or antenna. The following figure depicts the structure of a 4-layer **stack**:



The packets produced by each layer are generically referred to as **PDUs** (Protocol Data Unit). The PDUs of each layer have specific names such as **frame**, **datagram** or **segment**. These names must be used with precision in technical contexts.

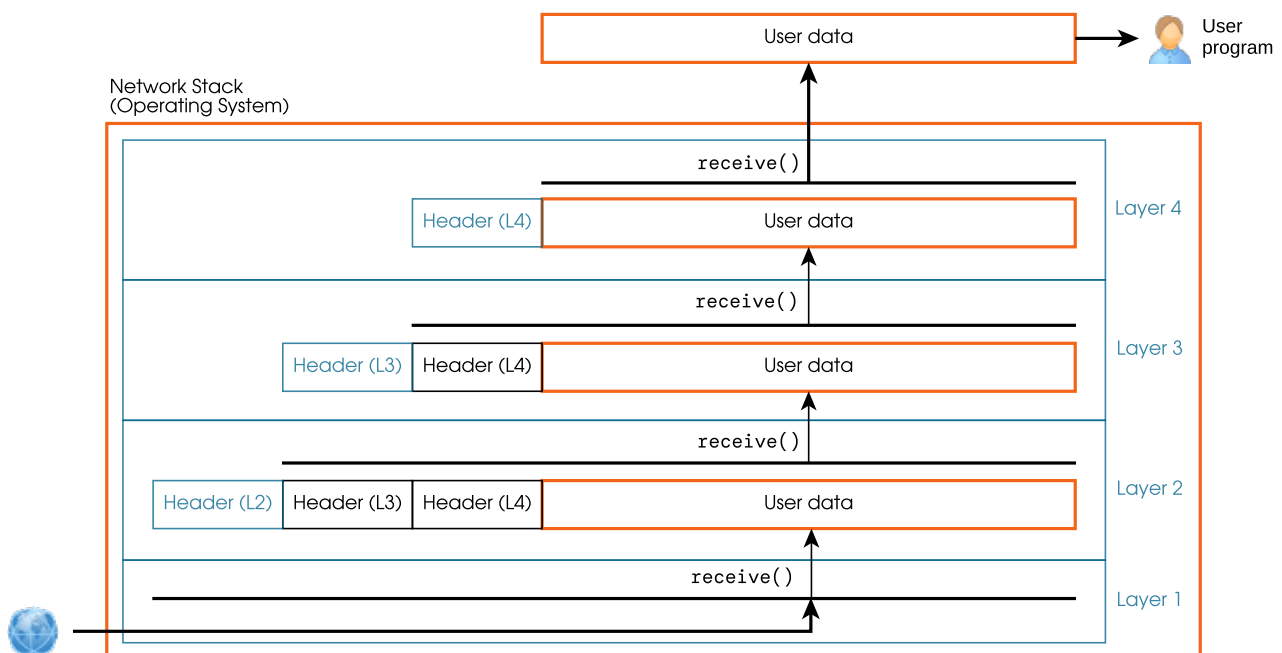
Each layer includes the **PDU** of the layer above inside its PDU's **packet format** definition. This process is referred to as **encapsulation**.

Exercise 5.1 The user program wants to send 420 bytes of data using the **stack** of the previous figure. Assume a header size for layers 2, 3 and 4 of 14 bytes, 20 bytes and 20 bytes, respectively.

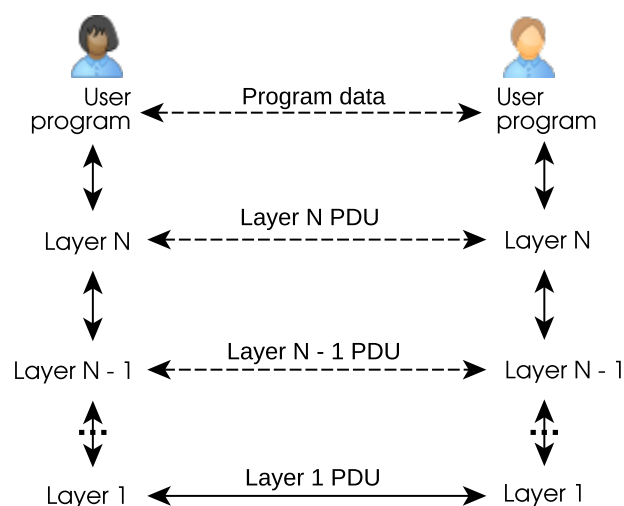
- What is the **payload** size and the **overhead** size for each **layer's PDU**?
- What is the **payload** size and the **overhead** size of the whole **stack**?

5.2 Receiving data

When a **packet** reaches the network **stack** of the recipient, it is processed in reverse order, *i.e.*, layers are traversed from bottom to top. Each layer inspects *only* the **header** corresponding to that layer. If everything is correct, the layer removes the header and passes the remainder of the packet to the layer above. In turn, the layer on top passes the remaining data (the **payload**) to the recipient. Continuing the example of the 4-layer stack:



In a **network stack**, only the bottom layer interacts directly with the exterior, *e.g.*, only the physical network card of a computer is connected to a wire that carries data to other computers. At the same time, data sent by the bottom layer includes the **PDU**s of all layers by means of **encapsulation**. Thanks to this, there exists a **virtual** communication between each layer in one end and its counterpart in the other end of communication. The following figure represents the real (solid line) and virtual (dashed line) communication in our example:



Exercise 5.2 Another network **stack** example has 3 layers: L1, L2 and L3 from bottom to top.

The **PDU** of layer L2 is as follows:

0	7	8	15	16	23	24	31
Destination address				Payload length (bytes)			
Payload (variable length)							
⋮							

The **PDU** of layer L3 is as follows:

0	7	8	15	16	23	24	31
Fragment offset				Payload length (bytes)			
Payload (variable length)							
⋮							

Layer L1 receives a **packet** of data and its `receive` function returns the following data (assume **unsigned** integers and **big endian** order when needed):

01 23 00 0a 01 00 00 06 00 aa 00 bb 43 20

- What data **payload** was sent to the recipient program in this **packet**?
- If this is the last **packet** of the communication, how much (**payload**) data was sent to the recipient user program?
- At most, how many different computers can connect using this **stack**?
- We send a file as the payload of this stack. What's its maximum possible size?

```

.py .out snippets/encapsulation.py
1  import typing
2  import struct
3
4  def layer2_encapsulate(address: int, data: bytes) -> bytes:
5      """Encapsulate a PDU of layer 3 into a PDU of layer 2."""
6      assert 0x0000 <= address <= 0xffff, "Invalid layer 2 address"
7      assert len(data) <= 0xffff, "Too much data for a layer 2 PDU"
8      return bytes(struct.pack(">H", address)
9                  + struct.pack(">H", len(data))
10                 + data)
11
12 def layer3_encapsulate(offset: int, data: bytes) -> bytes:
13     """Encapsulate a chunk of user data into a PDU of layer 3."""
14     assert 0x0000 <= offset <= 0xffff, "Invalid layer 3 offset"
15     assert len(data) <= 0xffff, "Too much data for a layer 3 PDU"
16     return bytes(struct.pack(">H", offset)
17                 + struct.pack(">H", len(data))
18                 + data)
19
20 def stack_send(data: bytes, output_file: typing.BinaryIO):
21     layer3_pdu = layer3_encapsulate(offset=0, data=data)
22     layer2_pdu = layer2_encapsulate(address=0xabcd, data=layer3_pdu)
23     output_file.write(layer2_pdu)
24
25 if __name__ == '__main__':
26     with (open(__file__, "rb") as input_file,
27         open("/dev/stdout", "wb") as output_file):
28         payload = input_file.read()
29         stack_send(data=payload, output_file=output_file)

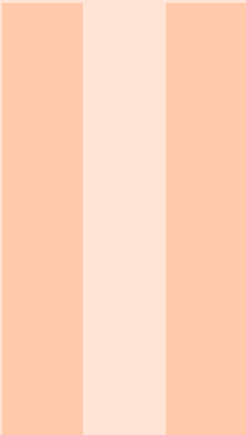
```

Exercise 5.3 The previous code takes some data and writes the bytes that would be sent to Layer 1 (the **physical layer**) of the **network stack** of Exercise 5.2. Implement the receiving end of the stack as follows:

- Implement a `layer2_decapsulate(data: bytes) -> (int, bytes)` function that takes a Layer 2 PDU and returns the tuple `(address, payload)`, where `address` is the destination address and `payload` is the encapsulated Layer 3 PDU.
- Implement a `layer3_decapsulate(data: bytes) -> (int, bytes)` function that takes a Layer 3 PDU and returns the tuple `(offset, payload)`, where `offset` is the payload offset, and `payload` is the chunk of data sent by the other end of communication.
- Implement a `stack_receive(data: bytes)` function that decodes a Layer 1 PDU (using the two previous functions) and prints a message line similar to
"Received a chunk of data. Length: 12 bytes, Address: 0xabcd, Offset: 0."
- Complete the `stack_send` function so that the input data are split in chunks of at most 100 bytes, a valid Layer 1 PDU is produced for each of those chunks, and those Layer 1 PDUs are passed to `stack_receive`.
- Would your implementation work if the Layer 1 PDUs were randomly ordered before passing them to the receiving stack?



Python's `struct` library is useful, but not mandatory, to format **packet** bytes.



Internet

The previous part of this guide explored some of the main challenges of communicating multiple computers. The **Internet** is one working solution to those challenges: the one humans have created over the last half century, now central to many social, commercial and political activities. This part studies the main features offered by the Internet, as well as the **suit of protocols** employed to achieve them.

6	The Internet	25
6.1	The TCP/IP stack	
6.2	TCP/IP protocol overview	
6.3	Who is the Internet boss?	
7	Layer 2: Network (LAN) communication	28
7.1	Layer 2 addressing	
7.2	Ethernet Layer 2 protocol	
8	Layer 3: Internet(work) communication	32
8.1	Layer 3 addressing	
8.2	IP – Internet Protocol	
8.3	IP routing	
8.4	IP subnetting	
8.5	IP fragmentation	
8.6	ARP – Address Resolution Protocol	
8.7	ICMP – Internet Control Message Protocol	
9	Layer 4: Transport	45
9.1	Layer 4 addressing	
9.2	UDP – User Datagram Protocol	
9.3	TCP – Transport Control Protocol	
9.4	TCP: “Connecting”	
9.5	TCP: “Connection established”	
9.6	TCP: “Closing connection”	
10	Layer 7: Application	56
10.1	DNS – Domain Name System	
10.2	DHCP – Dynamic Host Configuration Protocol	
10.3	Autonomous Systems	

6. The Internet

Modern communications are based on layered network **stacks**. Maybe you just downloaded this guide from the Internet. What layers are used exactly? What protocols are employed, and with what purpose? This chapter provides a primer to answer these questions.

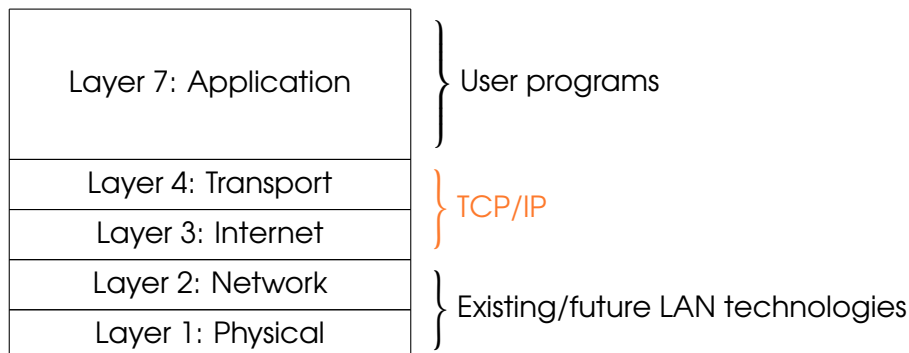
6.1 The TCP/IP stack

The **Internet** is a practical solution to the problem of global computer communication, whose development started in the late 1960's – early 1970's, and is still ongoing.

Packet switching was conceived during the early 1960s, and the first wide-area implementation, **ARPANET**, began in 1969. A key challenge at the time was to interconnect existing **networks**, owned by different organizations, and based on different technologies and **protocols**. To make things viable, all the following requirements had to be met:

- Existing networks should work without updates to the physical infrastructure or their protocols, just the way those protocols are used.
- Existing networks should remain independently managed at the local level (autonomous internal organization).
- Packets may not always be delivered, some are lost.
- Messages that arrive may not be in order, and may be repeated.

The solution adopted by ARPANET –and later by the whole world– is the **TCP/IP** protocol suite, first described in 1974. It is a layered approach, designed to be “plugged-in” on top of existing **LANs**, with the goal of offering networking capabilities to all user applications.



In a nutshell, the main services offered by these layers are as follows:

Layer 1 (Physical): Send **digital** data through **analog** media.

Layer 2 (Network): Identify and communicate devices within the same **LAN**.

The term “Data Link” layer (from the OSI model) is sometimes used.

Layer 3 (Internet): Identify and communicate devices across all **LANs**.

The term “Network” layer (also from OSI) is sometimes used.

Layer 4 (Transport): Identify and communicate programs (services).

Also (choose *only one*):

- a) Make sure a **stream** of data successfully reaches the other end, and in order
– *but* an **overhead** is introduced and **connections** must be established (**TCP**).
- b) Use simple, relatively efficient messages, and have a small overhead
– *but* messages may be lost, reordered or duplicated (**UDP**).

Layer 7 (Application): Do anything the user might need.

Don't forget about security (e.g., cryptography) or about efficiency (e.g., compression): Layers 4 and below don't provide that.

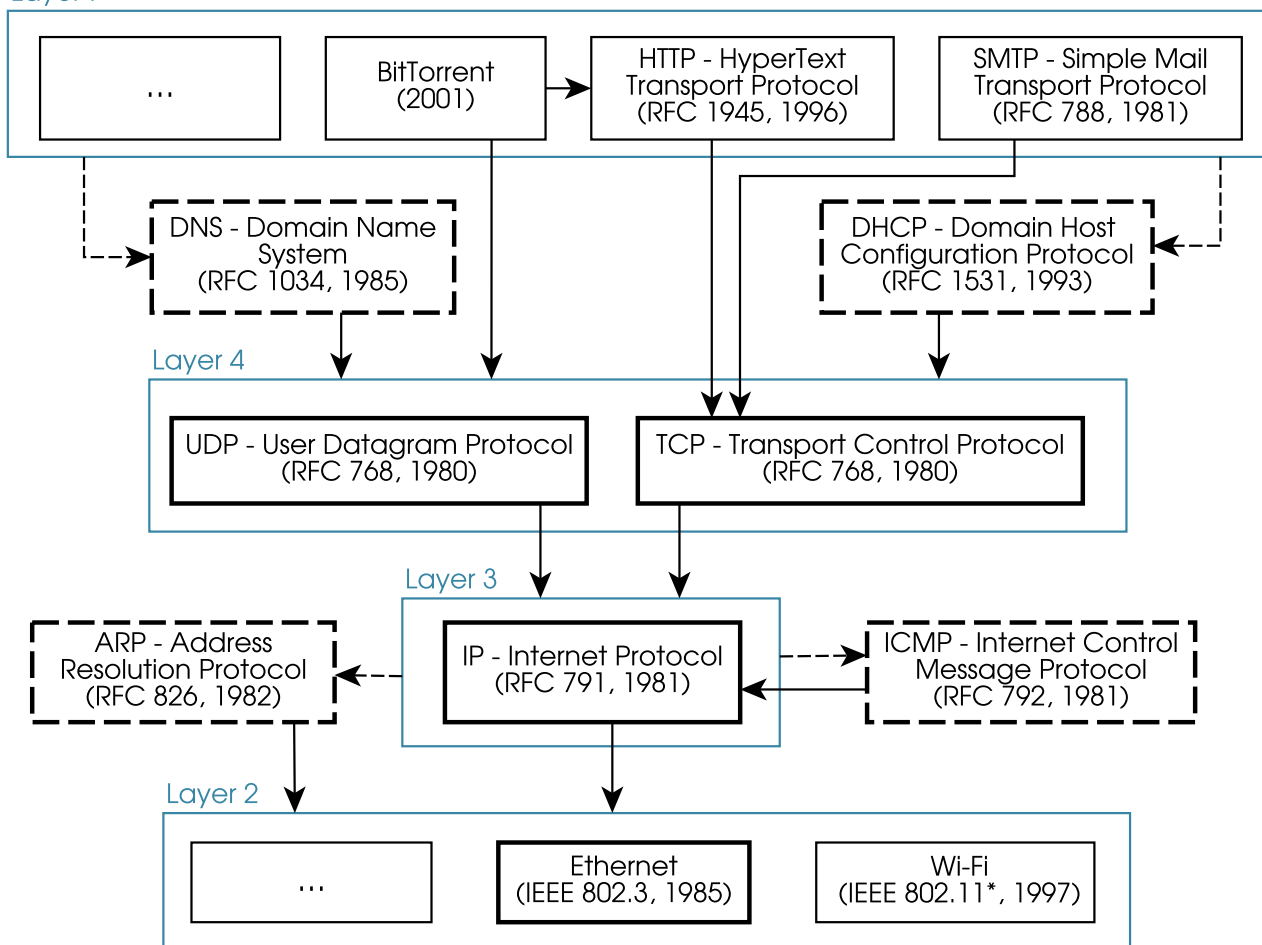
6.2 TCP/IP protocol overview

Multiple **protocols** are defined to implement the functionality of each **layer**. These definitions include the required **packet format**, the rules to exchange those packets and their semantics. These protocols must be understood as a system, in which each part has very specific tasks.

Some of the most relevant protocols to communications using the Internet are shown in the next figure. In it:

- Solid arrows indicate a protocol being encapsulated in another. If the destination is a layer, one protocol within that layer must be used.
- Dashed boxes indicate auxiliary protocols defined to help other function. Dashed arrows indicate what layer uses what auxiliary protocol.
- Boxes with thicker lines indicate the protocols highlighted in this guide.

Layer 7



Exercise 6.1 We want to send the sentence "Sorry, I'll be 5 minutes late" by email. Our email client will send those data using the SMTP protocol, which in turn uses TCP (part of Layer 4). Assuming SMTP uses headers in its **PDU**, how many headers will the corresponding Layer-2 **PDU** contain?

Exercise 6.2 Why do you think Layer 3 has only one protocol (two if you count IPv4 and IPv6 separately)?

6.3 Who is the Internet boss?

The **Internet Engineering Task Force (IETF)** was created in 1986 to develop and publish standards to be used in the Internet. Their main output are "**Request for Comment**" (**RFC**) documents, which contain complete technical descriptions of those standards.

These RFCs are created by working groups of experts, for everyone to use freely and free of charge. Anyone can participate in these working groups, including researchers, governments

and industry stakeholders. Decisions are made when 80-90% of the participants agree, so no single entity controls the standardization process.



"We reject kings, presidents, and voting. We believe in rough consensus and running code" — Dave Clark (IETF), 1992.

See DOI [10.1109/MAHC.2006.42](https://doi.org/10.1109/MAHC.2006.42) for a short story of struggle for power.

RFCs standards are descriptive, not prescriptive. This means that nobody polices the correct use of these standards. Instead, manufacturers have the incentive to be compliant with the RFCs so that their products are compatible with others. The value of RFCs (and any other standard) is dependent on whether they are adopted by users. RFCs can be updated and **retired** for this reason.

TCP/IP protocols prevail because they are flexible and work reasonably well in most situations, even though they are not optimal in virtually any of them. Also, TCP/IP protocols are free, as opposed to **privative** protocols that forced the purchase of a specific vendor's solution.



Today it is hard to imagine a world *not* dominated by the TCP/IP architecture, but entirely different stacks were commercialized and supported until the late 90s. Examples include AppleTalk and IPX/SPX, by Apple and Novell, respectively.

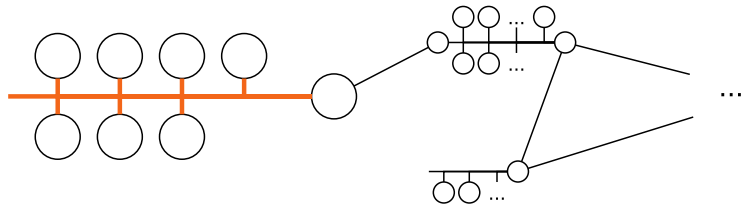
Some aspects of TCP/IP like unique identifiers (including **DNS**, the Domain Name System) and reserved numbers need to be agreed upon by everyone for the Internet to work properly. The **IANA** (Internet Assigned Numbers Authority) works in coordination with the **IETF** to define the appropriate **RFCs**.



Since 2016, IANA is managed by a nonprofit multistakeholder organization (**ICANN - Internet Corporation for Assigned Names and Numbers**). Since 2004, the responsibility for regional domains (e.g., **.cat**, **.fr**, **.es**) is transferred to continent-level organizations called regional Internet Registries (**RIRs**).

7. Layer 2: Network (LAN) communication

Layer 7: Application
Layer 4: Transport
Layer 3: Internet
Layer 2: Network (LAN)
Layer 1: Physical



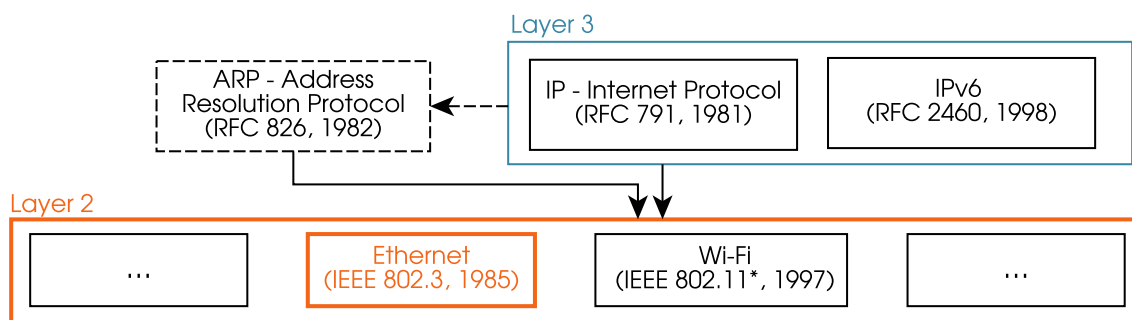
Capabilities

Layer 2 protocols allow multiple devices to exchange messages and identify each other within a local network (LAN). Devices outside the LAN cannot be contacted *directly* with Layer 2 protocols.

Layer 2 abstracts upper layers (including user applications) from the specifics of connected hardware. This makes it easier (read: cheaper) to write code once and run it everywhere.

Layer 2 messages are limited in length. The maximum amount of Payload data that can be inserted in a Layer 2 protocol is called the **Maximum Transmission Unit (MTU)**, e.g., 1500 bytes for Ethernet.

Some Layer 2 protocols may also provide user **authentication** and **cryptographic** protection; **Wi-Fi** is a notable example, although wired networks may also employ, e.g., IEEE 802.1X.



Protocols

Many **protocols** have been and are being defined within Layer 2. Most importantly nowadays:

- **Ethernet** (IEEE 802.3), addressed in Section [7.2](#).
- **Wi-Fi** (e.g., IEEE 802.11be aka Wi-Fi 7, of 2024).

Within **TCP/IP**, Layer 2 **encapsulates** the following protocol **PDUs** (both addressed in Chapter [8](#)):

- **IPv4** and **IPv6 datagrams**.
- **ARP** messages.



The `ip link` command shows and can configure your interfaces, including the **MAC** address.

Your Operating System typically assigns internal interface names automatically based on their type, e.g., `eth0` or `wlan1`. Interfaces can usually be renamed, because interface names are not part of TCP/IP; they are used exclusively within that Operating system.

7.1 Layer 2 addressing

In most LAN technologies, Layer 2 addresses are numeric IDs unique for each device within the LAN. The most frequent type are Ethernet MAC addresses (technically EUI-48) that span 6 bytes (*i.e.*, 48 bits). These are also used outside Ethernet, *e.g.*, by Wi-Fi and Bluetooth.

MAC addresses are most often presented to humans in the following format:

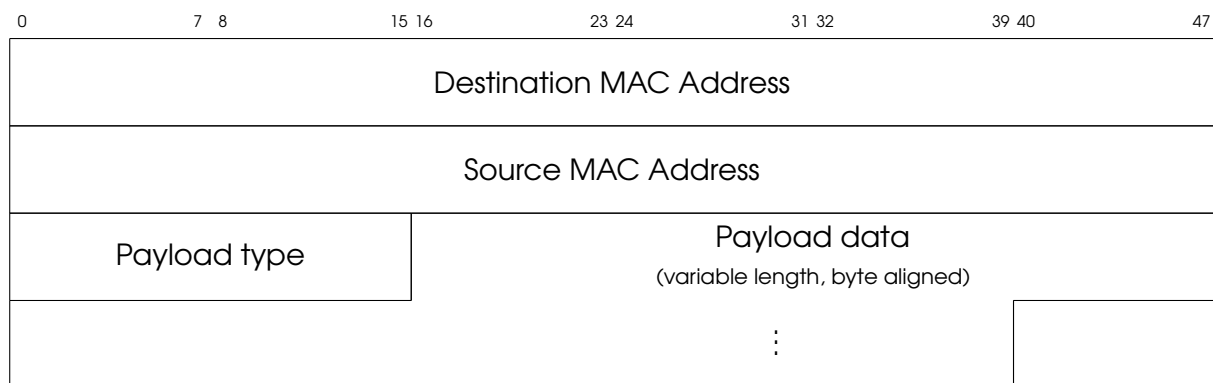
`01:23:45:67:89:ab`. Some addresses have a special meaning and are not valid device identifiers. For instance, `ff:ff:ff:ff:ff:ff` is the **broadcast** address, meaning “everyone in the LAN”. Also, `00:00:00:00:00:00` is used to represent an unknown MAC address (see [RFC 9542](#) for a full description of reserved MAC addresses, including **multicast**).

Network hardware comes with a predefined, unique built-in MAC address ready to be used. Device MACs can be configured at the OS level, but remain constant while in use.

7.2 Ethernet Layer 2 protocol

7.2.1 Packet format

Ethernet Layer 2 defines the following format, used in all messages. Packets of this type are called **frames**:



- **Destination MAC:** The MAC address of the destination device, or `ff:ff:ff:ff:ff:ff` for **broadcast**.
- **Source MAC:** The MAC of the device sending the frame.
- **Payload type:** Identifier for the protocol **encapsulated** in the payload, *e.g.*, `0x0800` → **IPv4**, `0x86DD` → **IPv6**, and `0x0806` → **ARP**. Sometimes called **EtherType**.
- **Payload:** Content requested to be sent, *e.g.*, by IPv4 or ARP. The maximum length of this field, the **MTU**, is 1500 bytes for ethernet.

Exercise 7.1

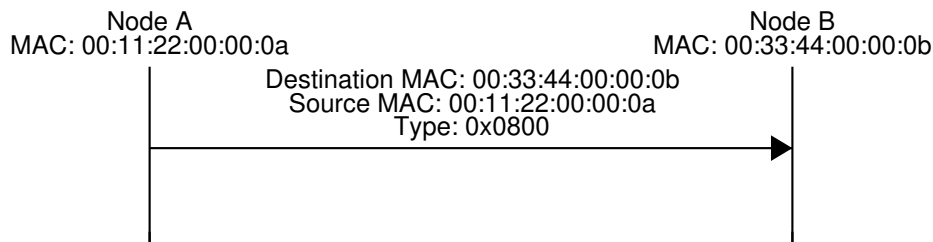
- How many different MAC addresses are there in Ethernet (including reserved **blocks**)?
- Are they enough so that every internet-capable device on Earth has a unique MAC?

Exercise 7.2 Is the following frame valid?

c2	21	90	15	bc	b6	42	40	e5	f7	5e	32	08	42	77	57
03	ca	27	6b	1f	97	e9	02	24	7f	80	fa	8b	61	59	2a
81	b6	73	b0	b4	e1	75	4f	ef	40	dd	9f	34	3c	48	67

7.2.2 Operation

Messages within an Ethernet LAN are normally sent in **unicast** mode, *i.e.*, they are one-to-one messages. This allows **switches** to minimize **power** and **bandwidth** consumption. In **unicast**, the source device node must know the MAC of the destination.



Ethernet also supports **broadcast** to all nodes in the LAN. In this case, the destination MAC address must be **ff:ff:ff:ff:ff:ff**. When a device receives a frame, it discards it unless the destination MAC field in the frame is identical to its own MAC address, or **broadcast**.

A node will also discard a frame with an unsupported Payload Type (**EtherType**) field. If the payload type is supported, it is used to decide what part of the OS receives the message, *e.g.*, the **IPv4** stack or the **ARP** subsystem.

Exercise 7.3 Continuing the example of the figure above, the network card of Node B receives a **frame** that begins with the following bytes. Should it be accepted by B?

00 11 22 00 00 0a ff ff ff ff ff ff 08 DD ...

Exercise 7.4 The following scripts exemplify how to send and receive raw **Ethernet** frames. The **client** sends 5 identical frames of **EtherType** **0x1234** and then exits. The **server** waits until it receives 5 Ethernet frames of that type (checked in lines 12-13) and then exits.

- Change the client's MAC address and interface name with yours.
- Make the server reject frames not addressed to it.
- Extend these scripts to implement a simple application with **LAN** capabilities.
Suggested examples:
 - A Layer-2 **echo** service that distinguishes between petitions and responses.
 - A Layer-2 **peer-to-peer** chat that includes the origin's nickname in all messages.
 - A Layer-2 calculator service that supports basic arithmetic operations (+, -, *, integer division //, optionally division /).
- What is missing so that your application can connect to the rest of the **Internet** beyond your **LAN**?

  snippets/ethernetclient.py

```

1 import socket
2
3 interface_name = "wlp5s0" # Needs manual configuration
4 source_mac = bytes([0x00, 0x11, 0x22, 0x33, 0x44, 0x55])
5 destination_mac = bytes([0xff, 0xff, 0xff, 0xff, 0xff, 0xff])
6 packet_type = bytes([0x12, 0x34])
7 payload_data = bytes([0x78, 0x6f, 0x69, 0x20, 0x75, 0x61, 0x62])
8
9 # This socket sends Layer 2 packets directly.
10 s = socket.socket(family=socket.AF_PACKET,
11                  type=socket.SOCK_RAW,
12                  proto=socket.ntohs(socket.ETH_P_ALL))
13 s.bind((interface_name, 0))
14
15 for i in range(5):
16     frame = destination_mac + source_mac + packet_type + payload_data
17     s.send(frame)
18     print(f"Sent frame {i + 1}/5: {len(frame)=}, {payload_data=}.")

```

  snippets/ethernetserver.py

```

1 import socket
2
3 interface_name = "wlp5s0" # Needs manual configuration
4
5 # This socket receives Layer 2 packets directly.
6 s = socket.socket(family=socket.AF_PACKET,
7                  type=socket.SOCK_RAW,
8                  proto=socket.ntohs(socket.ETH_P_ALL))
9 s.bind((interface_name, 0))
10
11 valid_frames = 0
12 while valid_frames < 5:
13     frame = s.recv(1514)
14     if frame[12:14] != bytes([0x12, 0x34]):
15         continue
16     valid_frames += 1
17     print(f"Received frame {valid_frames + 1}/5, {len(frame)} bytes. "
18           f"Payload: {frame[14:]!r}")

```



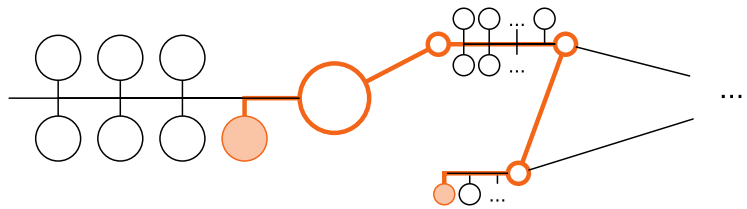
You will need to run these scripts with privileges, e.g., with `sudo`. Alternatively, you can permanently add the `CAP_NET_RAW` capabilities to your python binary with

```
sudo setcap cap_net_raw+ep ./venv/bin/python.
```

After that, you can run `./venv/bin/python script.py` directly without `sudo`.

8. Layer 3: Internet(work) communication

Layer 7: Application
Layer 4: Transport
Layer 3: Internet
Layer 2: Network (LAN)
Layer 1: Physical



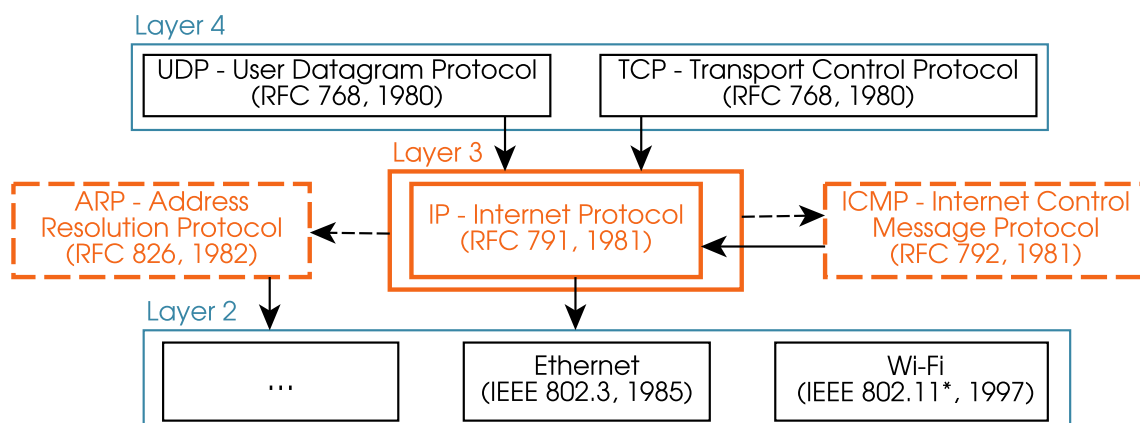
Capabilities

The **Internet Protocol (IP)** allows exchanging **datagrams** (Layer 3 PDUs) between devices from different **LANs**. Its addressing system identifies devices globally and makes it possible to **route** datagrams anywhere we need.

Devices communicating via IP may belong to LANs based on different technologies and with different types of **MAC** address. This has two implications:

- A datagram that “fits” in the **MTU** of the source LAN may be too large for the destination’s (or any intermediate’s) LAN MTU. IP defines a **fragmentation** system to split datagrams, which are reconstructed only at the final destination.
- When we send a datagram, we know the target device’s **IP address**, but not necessarily its **MAC** address. Only if we are part of the destination LAN we will need that MAC. IP uses **ARP** to translate known IP addresses to MAC addresses of the destination LAN’s type.

Datagrams may travel through networks not controlled by neither the source nor the destination, so datagrams may be lost, reordered and duplicated. Layer 3 does not provide mechanisms to deal with this; upper layers must implement protection mechanisms when required (e.g., to implement **TCP’s data streams**).



Protocols

Version 4 of **IP** is the protocol used in Layer 3, i.e., it is common to virtually all communications in the Internet as we know it today. For many years, the Internet has been struggling to transition towards **IPv6**, but the process is not complete and only **IPv4** provides global coverage.

IPv4 delegates in the **ARP** protocol the translation of known IP addresses into MAC addresses within each LAN. ARP does *not* use IP datagrams.

The following protocols encapsulate their PDUs in the payload of IP **datagrams**:

- Transport Control Protocol (TCP)
- User Datagram Protocol (UDP)
- Internet Control Message Protocol (ICMP)



The `ip address`, `ip route` and `ip neighbor` commands (among others) let you query several aspects of your IP configuration.

8.1 Layer 3 addressing

IPv4 addresses are 32 bits long. They are most often presented to humans in the **quad decimal** format, e.g.,

142.250.201.67

IPv6 addresses are 128 bits (12 bytes) long, i.e., they are 4 times larger than an IPv4 address. They are expressed in hexadecimal format, e.g., as follows. Note how two colon signs “::” indicate “fill with zeros”, and “::” may only appear once per IPv6 address.

fe80::f524:a73a:e946:7c02



When an IPv6 address is used as part of a URL, it is surrounded with brackets, e.g., `https://[fe80::f524:a73a:e946:7c02]/`.

8.1.1 Public and reserved addresses

Most IP addresses are **public** and unique across the Internet, i.e., the previous IP has the same meaning worldwide: it identifies a single connected device. Many addresses are **reserved/private** and can only be used within a particular scope (e.g., a LAN) or situation. Some of the most common are the following (there are [some more](#), also for IPv6):

IP address block	Address scope
127.*.*.*	This computer (localhost)
10.*.*.*	This LAN
172.16.*.*	This LAN
192.168.*.*	This LAN
0.*.*.*	Special usages
169.254.*.*	Special usages
255.255.255.255	Special usages

Exercise 8.1

- How many different IPv4 addresses are there (including reserved and private)?
- Are they sufficient now and in the future?
- Is it problematic that an address like 192.168.0.1 is simultaneously used by thousands (likely millions) of computers in the Internet right now?
- Which of these corresponds to **link local** addresses?

8.1.2 Netmasks

The 32 bits of an IP address can be divided in two parts using a **netmask**:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Network ID										Device ID																					

The first part identifies the LAN or group of LANs to which a device belongs. The second part identifies a single device in the LAN. The netmask determines how many bits are assigned to each part.

In the previous figure, the netmask assigns 10 bits to the network ID and 22 to the device ID. This mask can be anywhere from 0 (all bits are device ID) to 32 (all bits are network ID).

Netmasks are presented to humans in two main ways:

- $/N$, where N is the number of bits assigned to the Network ID. This is used in the **CIDR** format.
- The **quad decimal** (**A.B.C.D**) format of N bits set to **1**, followed by $(32 - N)$ bits set to **0**.

Exercise 8.2 The netmask of the previous example can be expressed in binary (although it is not very convenient):

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10 bits										$32 - N = 22$ bits																					

- Express the netmask of the previous example in the CIDR and quad decimal formats.
- What is your IP and netmask? Express it in the same two main ways.



Netmasks are used in IPv6 as well, but with a maximum value of N of 128 instead of 32.

8.1.3 Blocks of IP addresses

Netmasks are used in combination with IP addresses to define **blocks of IP addresses** that begin with the same N bits (where N is the netmask). For instance, **192.168.3.0/24** (in **CIDR** format) and **192.168.3.0/255.255.255.0** refer to the block of addresses between **192.168.3.0** and **192.168.3.255**, both included (**192.168.3.*** in the informal notation of the previous table).

Exercise 8.3 IP blocks can be smaller than, equal to or larger than an IP network. Provide examples where an IP block refers to:

- A single IP
- Half the IP addresses in the **192.168.3.0/24** network.
- All addresses of the **192.168.3.0/24** network, plus 256 more addresses.
- All possible IPs.

Exercise 8.4

- Can every **netmask** be expressed as an **IP address**?
- List all possible netmasks.

When referring to a block of addresses (like a LAN), the IP address in this IP + netmask combination must have all device ID bits set to **0**. This way, **192.168.3.0/24** correctly refers to a network, but **192.168.3.1/24** refers instead to a device (the one with IP **192.168.3.1**) in that network (**192.168.3.0/24**).

8.1.4 LANs and netmasks

Netmasks need *not* be multiples of 8. Consider the following example in which **192.168.76.1/19** (a device's IP + netmask) is used to obtain its network address (**192.168.64.0/19**):

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
192								168								76								1							
1	1	0	0	0	0	0	0	1	0	1	0	1	0	0	0	0	1	0	0	1	1	0	0	0	0	0	0	0	0	0	1
& &																															
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
= =																															
1	1	0	0	0	0	0	0	1	0	1	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
192								168								64								0							

Device's IP

Netmask

Network IP

} Device's IP

} Netmask

} Network IP

Exercise 8.5

- In the previous example, we only really needed to transform one decimal value to binary, and one decimal value to binary. Which one?
- Is this always true when converting IP+netmask combinations to network addresses?
- Express the netmask in **quad decimal** format.

Given a IP + netmask address block and a device's IP address, it is direct to determine whether that device's IP belongs to the block (if and only if the first N bits match, where $/N$ is the CIDR netmask).

Exercise 8.6 Determine whether each of the following addresses belong to the same network as the following device **123.45.67.89/14**:

- **23.45.67.89**
- **123.45.203.25**
- **123.43.255.255**
- **0.45.67.1**

An **IP network** (a LAN) is a block of addresses, but not all those addresses can be assigned to devices. Two addresses are always reserved:

- The address with all node bits set to **0**: it identifies the LAN itself.
- The address with all node bits set to **1**: it identifies **broadcast** within the LAN.

Exercise 8.7

- List all addresses that can be assigned to devices in **192.168.54.0/29**.
- List the broadcast address of that network.

The netmask $/N$ determines the number of devices that can be connected at the same time to a network: $2^{32-N} - 2$. When LANs are called "small" or "larger", the size refers to that number of available device addresses.

Conversely, the number M of devices that we need to connect to the same LAN determines the smallest LAN size (largest netmask $/N$) that fits all those devices at the same time:

Netmask $/N$	Total addresses 2^{32-N}	Max devices $2^{32-N} - 2$
/30	4	2
/29	8	6
/28	16	14
/27	32	30
/26	64	62
/25	128	126
$\vdots$	$\vdots$	$\vdots$



All **routers** need a valid IP address per LAN to which they are connected. Therefore, routers do count towards the maximum number of devices connected in a LAN.

Exercise 8.8

- What's the minimum network size that fits 256 connected devices?
- What's the longest netmask $/N$ that would let everyone in your city connect?
- How many addresses are contained in a **/0** network?
- How many **/0** networks may exist? Are they given names?

Exercise 8.9 The following code snippet takes an IP address and shows its 4 bytes expressed in quad decimal and in binary. Extend (or replace) the code so that you can:

- Transform 32 bits into a quad decimal string.

- Display a netmask in IP address and CIDR formats.
- Given a device's IP and netmask, determine whether other IP belongs to the same network.

  snippets/ipcalculator.py

```

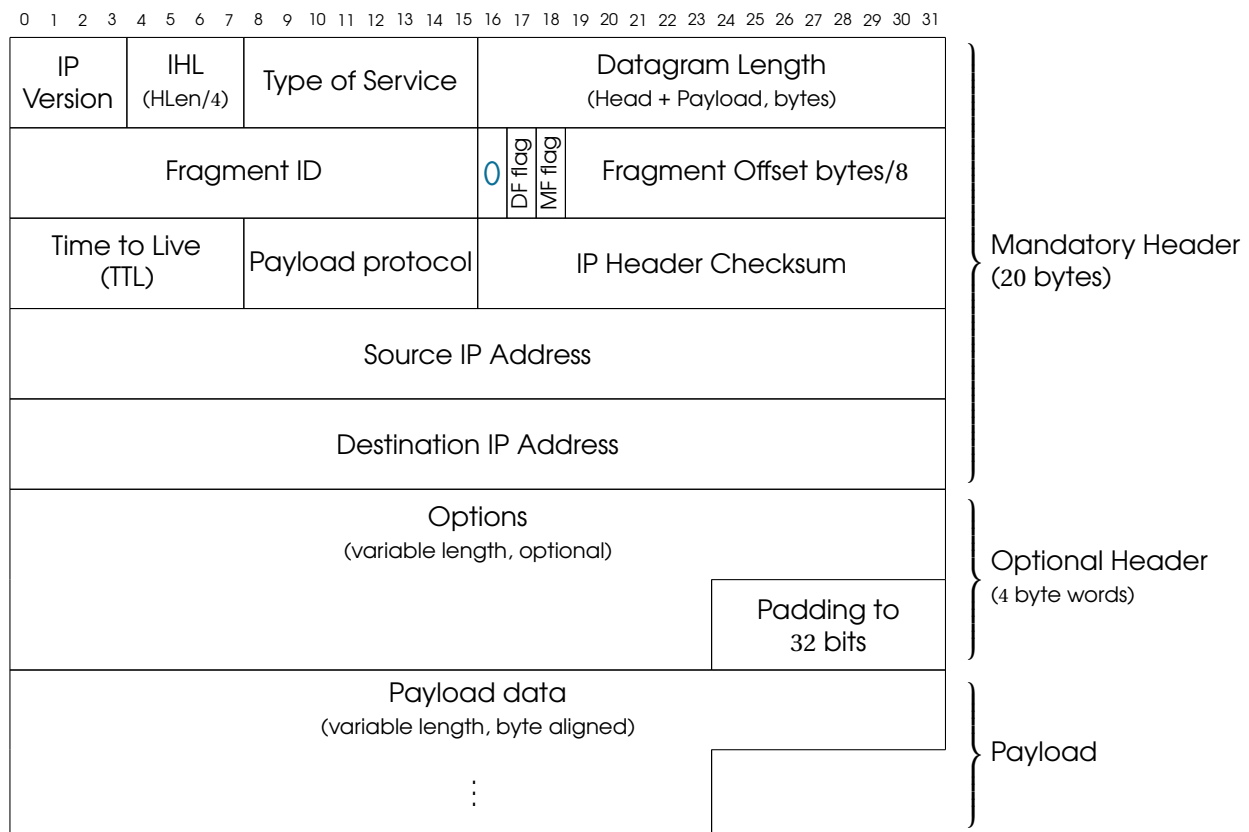
1 def print_ip_binary(ip: str):
2     parts = [int(s) for s in ip.split(".")]
3     print(" ".join(f"{part:8d}" for part in parts))
4     print(" ".join(f"{part:08b}" for part in parts))
5
6 print_ip_binary(ip = "192.168.0.1")

```

8.2 IP – Internet Protocol

8.2.1 Packet format

IP defines a **packet** format with a **header** that is *at least* 20 bytes long. Optional parts may be included, as long as the total header size is a multiple of 32 bits (4 bytes). The payload can be empty, so the minimum datagram size is 20 bytes.



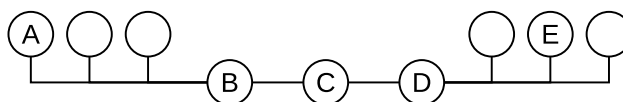
Exercise 8.10

- How can the payload length be calculated from the header?
- What is the maximum payload length?
- What are the possible values of the flags in the IP header?
- What's longer, an IP address or an Ethernet MAC address?
- Is it possible to send exactly 17 bits of payload data in an IP datagram?
- How can we know whether a datagram is encapsulating TCP or UDP data?
- Why must the optional header part be a multiple of 32 bits?

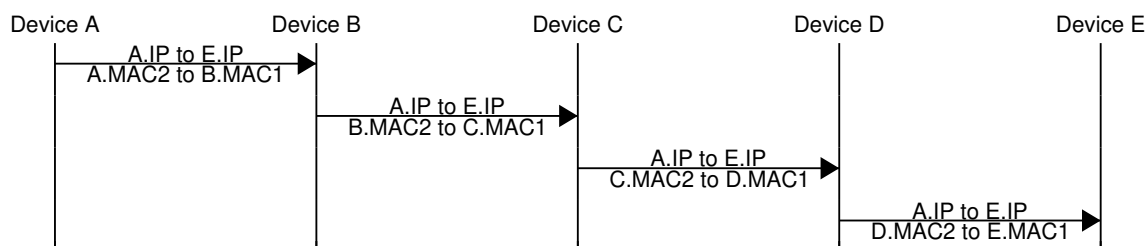
8.2.2 Operation

When a device A wants to send an IP datagram to a device B, the source and destination IPs in the header are fixed and don't change for this datagram.

If devices A and B belong to the same LAN, the datagram is sent directly (e.g., encapsulated in an Ethernet frame). However, IP can go much farther. Consider the following scenario, where A wants to communicate with E.



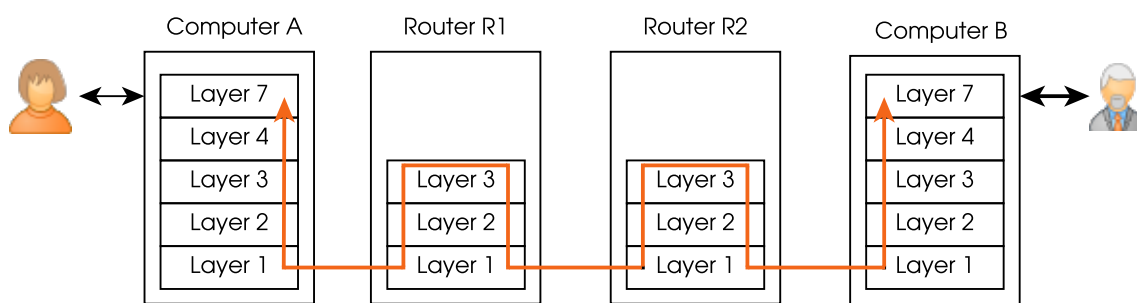
Here, the source and destination devices belong to different LANs. The datagram is passed first to the source's **router**. Then, each intermediate router passes it to the next one until the datagram reaches the last router, which is connected to the destination LAN. Finally, that last router passes the datagram to the destination.



The source and destination IP addresses don't change throughout the journey. In contrast, MAC address change in each LAN **hop**.

8.3 IP routing

Routers are the devices in charge of forwarding IP **datagrams** by looking at the destination IP address of each one, present in the IP **header**. Routers don't need to **decapsulate** headers of layers 4 and above. Therefore, their network stacks need contain only up to Layer 3 (and so are sometimes referred to as Layer-3 devices).



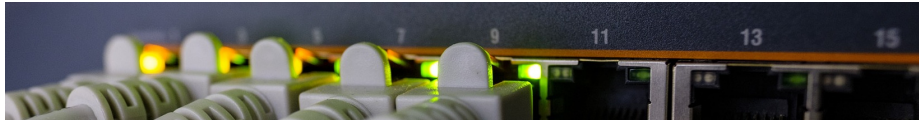
When a router detects a network frame F in one of its **interfaces** (network cards), it tries to recognize the following parts of a datagram.

Layer 2 header	IP header	Datagram payload
----------------	-----------	------------------

If recognized, the router processes the datagram as follows:

1. The whole frame F is discarded unless the destination MAC (in the Layer 2 header) is that of the router's interface that detected the frame.
2. The router looks at the destination IP address (in the IP header of F) and at its **routing table** to decide the next **hop** of the datagram, i.e., the device to which this router will pass the datagram to, and what **interface** to use (see Section 8.3 below).
3. The router creates and sends a frame F' making the following two changes to F :
 - a) The Layer 2 header in F' changes entirely:
 - The destination **MAC** is that of the next hop, and the source MAC is that of the router forwarding the datagram. The next hop's IP is given by the **routing table** (see Section 8.3), and its MAC is obtained from that IP using **ARP** (Section 8.6).

- The source MAC of F' frame is different from the destination MAC of the received frame F if the router received F in one interface (e.g. `eth7`) and forwards it through another (e.g., `eth9`).



b) The IP header in F' differs from that of F in the following:

- The **TTL** (time to live) field is decreased by 1.
If the TTL reaches 0, the datagram is discarded instead of being forwarded.
- When the **Options** field is present, it is often updated.
- The header **checksum** is updated accordingly.



Note how the IP addresses and the payload are copied over unmodified when a datagram is forwarded.

When a router receives an IP **datagram** to forward and that router is connected to the same LAN as the destination IP address, the router performs a **direct delivery** (DD). Otherwise, the datagram is forwarded to the next router in an **indirect delivery** (ID). A router knows whether to perform DD or ID, as well as the destination (and source) IP address to use, thanks to its **routing table**.

All route tables have at least the following columns:

- **Destination address block:** one or more IP addresses that can be expressed as an IP + netmask combination. A row in the table may apply to a datagram only if its destination IP is within that row's destination address block:
 - If only one row of the table applies, that row is used for routing.
 - If more than one row applies to a datagram, the row with the smallest destination address block (the most specific) is used.

A destination block of **default** indicates the `0.0.0.0/0` block. Since this is the largest block possible, it is only applied if none of the other rows apply. The **default** entry is not mandatory.

- **Gateway/via:** if the row corresponds to an **indirect delivery**, this column contains the IP address of the next router. Otherwise, this column contains `0.0.0.0` to indicate a **direct delivery**.
- **Interface:** internal name of the network interface (e.g., of the Ethernet card) to use to deliver the datagram. Operating Systems may use any convention they wish, as long as they can be uniquely identified in the table. Common names begin with `eth`, `wlan`, `atm`, and more recently `en` and `wl`.



All devices have their own routing table, not just routers.

Exercise 8.11 The **routing table** of a small router is presented next.

Destination address block	gateway/via	Interface
<i>default</i>	192.168.0.1	eth0
192.168.0.0/24	0.0.0.0	eth0
172.16.0.4/32	0.0.0.0	eth1
10.49.4.0/28	10.49.4.1	eth2
10.49.4.3/32	10.49.4.2	eth2

- Draw a network map consistent with the routing table, containing this router. For the router's interfaces, indicate its name and some valid IP and Layer 2 addresses.
- How would this router forward a datagram directed to a **public** IP address?

- Provide a destination IP address that would be processed using the `10.49.4.0/28` row, and another that would be processed using the `10.49.4.2/32` row. Think of a use case for having these two rules.

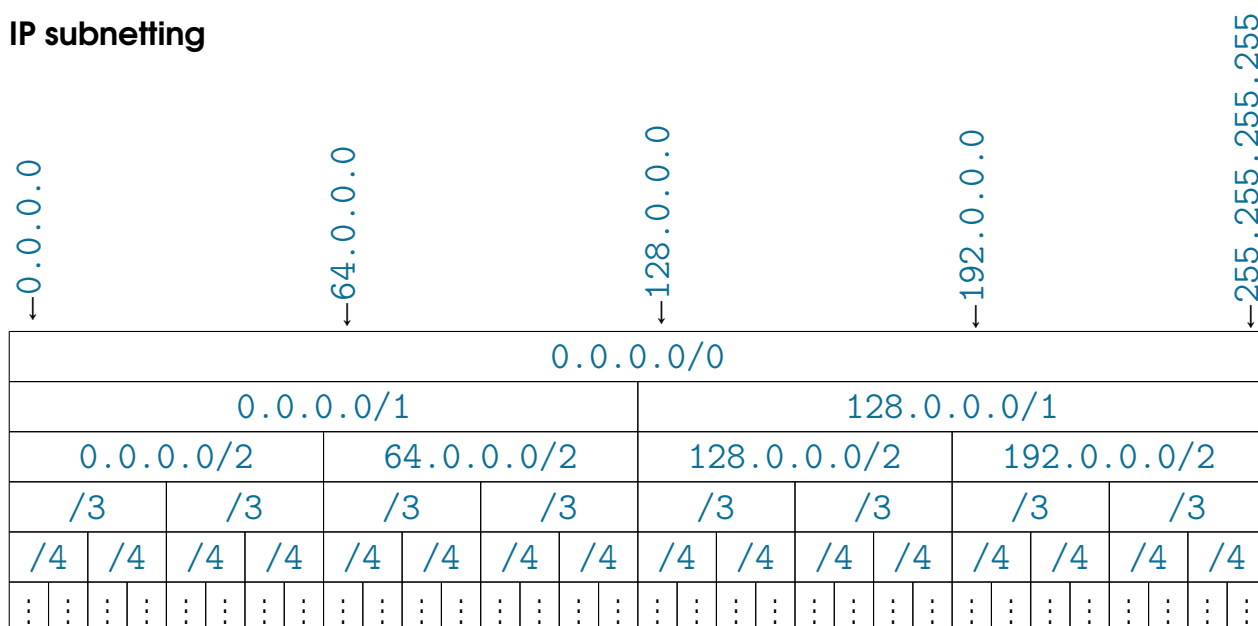
For the Internet to work, routers interconnecting LANs globally must be well configured. This is sometimes not the case, which can cause **routing errors**. Two important problems one can encounter are:

- “Network is unreachable”: none of the rows in the routing table uses an address block that contains the datagram’s destination address. The router cannot know what the next hop should be, so the datagram is not forwarded. This causes an “ICMP Destination Unreachable” error (see Section 8.7).
- “Time to Live exceeded”: every time a datagram is forwarded, its **TTL** is decreased, as mentioned in Section 8.2.1. If several routers are configured so that the datagram follows some form of cycle (loop), the datagram would be infinitely forwarded. When the **TTL** reaches 0, the datagram is discarded and this error is produced. This causes an “ICMP Time Exceeded” error (see Section 8.7).

Exercise 8.12 Can the “Network unreachable” error happen if a router has a *default* entry?

Exercise 8.13 Propose an example of interconnected LANs, and specify the necessary routing tables so that a datagram would follow a cycle and yield the “TTL exceeded” error.

8.4 IP subnetting



LANs can be regarded as address blocks (e.g., expressed in **CIDR** format) with two reserved addresses. Networks directly accessible through the Internet must be assigned a valid block large enough to connect all required devices. To make sure no two devices in the Internet have the same **public** IP, no two LANs can contain the same address (unless the address is **private**).

The 2^{32} addresses available in **IPv4** are divided in blocks by means of **netmasking**. Larger blocks can be divided in half, by increasing the netmask length by 1, as seen in the figure above. For instance, the initial block `0.0.0.0/0` (all addresses) can be split into two `/1` blocks, each with half the **IPv4** addresses (2^{31} – still too large for any real LAN). This process continues independently for each resulting blocks, and stop when the resulting LAN size is desirable.

Given a `/N` address block (e.g., from a single existing LAN), we may want to divide it into subblocks and have several smaller LANs (**subnets**) instead. We can only use address from the

original block in the subnets, but blocks can be reserved for future use:

Exercise 8.14 You are the administrator of the $192.168.0.0/24$ network in a research lab. You would like to split into the three subnets highlighted below.

- Provide a diagram of the three networks, including a router connected to all three via three different interfaces.
- Provide the *minimal* routing table of that router (these networks are not connected to the Internet).
- How many devices can be connected in total to these three networks, including routers?
- How many addresses from the original network are still unassigned?
- What is the largest LAN we can create with the unassigned addresses?

$192.168.0.0/24$							
$192.168.0.0/25$				$192.168.0.128/25$			
$192.168.0.0/26$	$192.168.0.64/26$	$192.168.0.128/26$	$192.168.0.192/26$				
$/27$	$/27$	$/27$	$/27$	$/27$	$/27$	$/27$	$/27$

The process of creating multiple smaller LANs from a single address block is called **subnetting**. The inverse process, combining multiple LANs into a larger one is called **supernetting**.

Two $/N$ LANs can be combined if and only if their network addresses (their IP without netmask) share the first $N - 1$ bits. This process can continue all the way up to a $/0$ network provided we own the IP addresses of both halves of each supernet.

Exercise 8.15

- What other network can you combine with $192.168.0.0/24$ to obtain a $/23$ network $N_{/23}$? What is its network address?
- $N_{/23}$ is part of exactly one $/22$ network, $N_{/22}$. List all $/24$ networks you would need to own to combine them into $N_{/22}$.

Subnetting is often based on how many IP addresses should be available in each subnet. That size determines the number of bits subnet's netmask as seen in Section 8.1.4. Additional constraints can be added, e.g., making one subnets IP addresses the largest, or smaller than other subnets.



Consider for example the $192.168.0.0/16$ network. We want to subnet this network and create the smallest network that can connect 100 devices simultaneously.

As seen in Section 8.1.2, this determines that if a subnet must be created from a to hold at least 100 nodes, at least 7 address bits ($2^6 - 2 = 62$, $2^7 = 126$) must be devoted to device IDs and the remaining $32 - 7 = 25$ to identify the network, i.e., the subnet can be as small as a $/25$.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
192.168																Free bits							Device ID								

The remaining 9 bits (positions 16 to 24) can be freely set if we don't have any further constraint. Choosing a value of these bits means choosing one of the 2^9 networks of size $/25$ contained in the original net, $192.168.0.0/16$. Setting those free bits to 0 yields the lowest IP range, and setting them to 1 the highest.

When multiple subnets need be created, it is essential to verify that no subnet is contained in another. Otherwise, the same IP address would belong to two LANs, and routing would be impossible. For instance, if three subnets for 100, 500 and 200 devices must be created from $192.168.0.0/16$, with address ranges in that order, one possible division would be:

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

192.168	00000000	Device ID	→ 192.168.0.0/25
192.168	00000001	Device ID	→ 192.168.2.0/23
192.168	00000001	Device ID	→ 192.168.1.0/24

Recall that the network address of each resulting subnet has all Device ID bits set to 0, and that this address is most often presented in **quad decimal** format.

Exercise 8.16

- In the previous example, why are the free bits of the second subnet **0000001** and not **0000000**?
- Repeat the exercise assigning the highest possible addresses to the third, then the next highest possible address to the second subnet, and the lowest possible address to the first subnet.
- What is the **broadcast** IP in each subnet?

8.5 IP fragmentation

Before a device (including **routers** and regular computers) can send a **datagram** through a LAN, it needs to verify that its length in bytes is not larger than that LAN's **MTU**. Otherwise, the datagram needs to be **fragmented** into two or more smaller datagrams that do fit in the LAN's MTU.

The smaller fragments are complete datagrams with header and payload. Therefore, more fragments imply a larger **overhead**. Each of these fragments is sent and forwarded individually like any other datagram and may need to be refragmented if it passes through another LAN with even smaller MTU.

When a datagram needs to be fragmented, the following steps are followed by the network stack:

1. The **DF** (don't fragment) flag of the original datagram is checked. If **DF=1** but fragmentation is needed, the datagram is not fragmented and an error is produced.
2. **Payload** data are divided in chunks of size at most S , so that $S + H = MTU$ (H is the datagram header length, typically 20 bytes). Only the last chunk may have size less than S . This guarantees that no fragment exceeds the MTU, which is the very purpose of fragmentation.
3. The original datagram's **header** is copied into the fragments' with the following modifications:
 - The Datagram ID field is set to the same value in all fragments of a datagram. The OS makes sure to pick an ID that has not been recently used by its network stack. The Datagram ID field is ignored for non-fragmented datagrams.
 - The **Offset** field in each fragment indicates the position (byte index) of its first payload data byte in the datagram's payload before fragmentation. This field is not expressed in bytes, but in 8-byte words. Given an offset in bytes, the **Offset** field is calculated dividing by 8. Therefore, fragment offsets can only be multiples of 8.
 - The **MF** (more fragments) flag is set to **1** (enabled) in all fragments except for the last one. This needs to be true even if one or more fragments are re-fragmented in a successive LAN. Thus, when fragmenting a datagram D with **MF=1**, none of the resulting fragments
4. Fragment datagrams are sent the same way a regular datagram would. Fragments may be re-fragmented, but are only re-assembled by the destination host.

Exercise 8.17 Given the **Offset** and the **MF** flag, how can we differentiate a datagram that has not been fragmented from a datagram fragment?

Exercise 8.18 Why would it be problematic to set **MF=0** in the last datagram produced by fragmenting a datagram with **MF=1**?

Exercise 8.19 A device needs to send a datagram with IP header size 20 bytes and payload length 2170 bytes. The device is connected to LAN1, and the datagram needs to traverse LAN2 and LAN3 until it arrives to the destination LAN4. The MTUs of these LANs is as follows:

LAN	LAN1	LAN2	LAN3	LAN4
MTU	1500	710	4096	864

- Provide the offset and payload length (both in bytes), as well as the MF and DF flags of the 4 datagrams that arrive to LAN4 as a result of sending the datagram (assuming no losses or duplications).
- Can the source device know the MTUs of the intermediate and destination LANs?

Exercise 8.20 Datagrams containing fragments may arrive duplicated and/or reordered. How should the receiving network stack deal with these datagrams?

8.6 ARP – Address Resolution Protocol

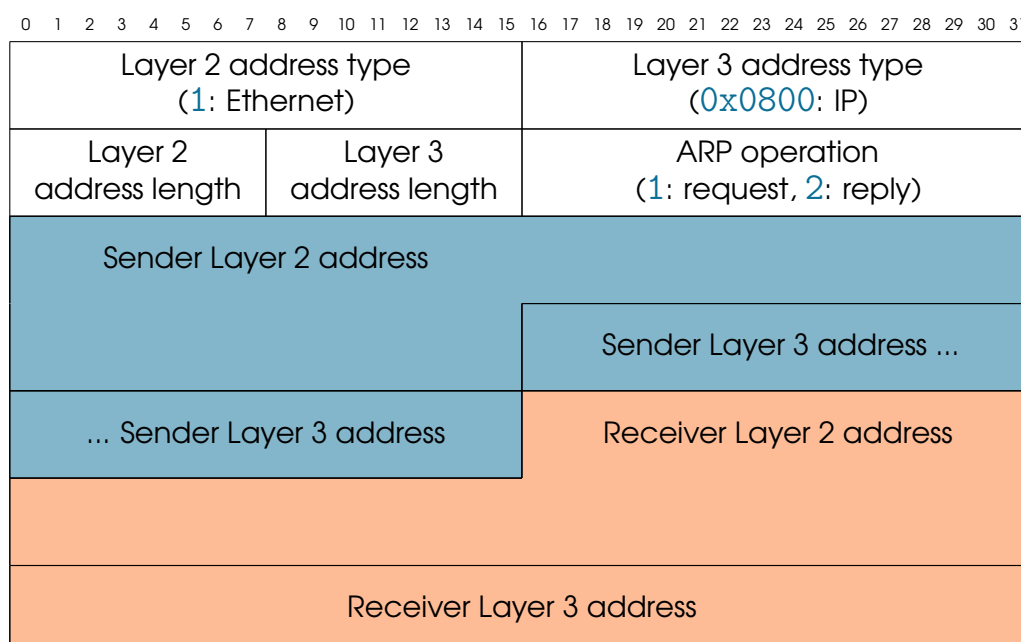
The Address Resolution Protocol (**ARP**) is used by Layer 3 (**IP**) to obtain the destination **MAC** (physical) address corresponding to a known IP address.

ARP packets are sent as the payload of Layer 2 (e.g., Ethernet) frames. ARP packets are *not* datagrams and do *not* contain IP headers.



The IP queried by the Layer 3 is that of the **router** for **indirect delivery** and of the destination device for **indirect delivery**. Also remember that the datagram's destination IP does not change during forwarding.

8.6.1 Packet format



The **ARP** protocol does *not* use IP headers.



"Sender" and "receiver" should be interpreted as "... of this ARP message".

Exercise 8.21

- What is the purpose of the first 4 fields of an ARP packet?
- What effect do they have in the remaining fields?

Exercise 8.22 How many valid addresses appear in an Ethernet frame carrying an ARP request? And in an ARP reply?

8.6.2 Operation

When a node (A) needs to translate an IP address into a MAC address, it sends a **broadcast** frame (destination MAC `ff:ff:ff:ff:ff:ff` containing an ARP packet. In that ARP packet, the operation is set to REQUEST (1), the destination IP is the one being translated, and the destination MAC is set to `00:00:00:00:00:00`.

When a network stack (B) sees an ARP request packet, it sends an ARP reply packet only if its configured IP is that of ARP's destination IP address. That reply is identical to the request with these changes:

- The OP code of the ARP packet is set to REPLY (2).
- "Source" now refers to (B) (it referred to (A) in the request), and "destination" to (A).
- All 4 addresses of the packet are filled in a reply (they may not be `00:00:00:00:00:00`).

Exercise 8.23 When the `ip route` command is executed in a node, it contains:

```
default via 192.168.0.1 dev wlp5s0 src 192.168.0.2 metric 600 [...]
```

In turn, the `ip link` command contains the following lines:

```
2: enp42s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 state DOWN [...]
   link/ether d8:43:ae:61:ed:a4 brd ff:ff:ff:ff:ff:ff
3: wlp5s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP [...]
   link/ether 5c:e4:3a:18:97:1b brd ff:ff:ff:ff:ff:ff
```

- The node needs to perform the **indirect delivery** of a datagram, but does not know the MAC address of its gateway router. Indicate the value of all fields (including Ethernet header and ARP message) of the ARP request that the node will send to get the router's MAC.
- Describe the field values of the ARP reply (you can use any valid address for the router's MAC address).

The MAC address associated to an IP is not likely to change very often. When a valid ARP response is received, the OS updates a **cache** adding (or updating) the received IP → MAC translations. If the OS needs to send a datagram to that IP before that cache entry expires, the table's MAC address is used directly without requiring additional ARP messages. The `ip neighbor` command shows the contents of that table, e.g.,

```
192.168.0.1 dev wlp5s0 lladdr cc:2d:14:fc:22:60 REACHABLE
```



ARP can be used to **scan** for used IPs in a LAN. In particular, a node can use item ARP to verify that its configured IP address is not being used by anyone else.

ARP was proposed at a time where network security was not an issue yet. As a result, it does not provide any mechanism to verify that a given ARP reply is correct. This can be exploited to **poison** a device's ARP cache table by sending **gratuitous** (unsolicited) ARP replies to the victim with phony IP → MAC associations.

Modern OSs will typically ignore unsolicited ARP replies, and modern **switches** will filter ARP messages not matching the actual IP and MAC addresses. Notwithstanding, IPv6 ditches ARP in favor of the more complex Neighbor Discovery (ND) protocol.

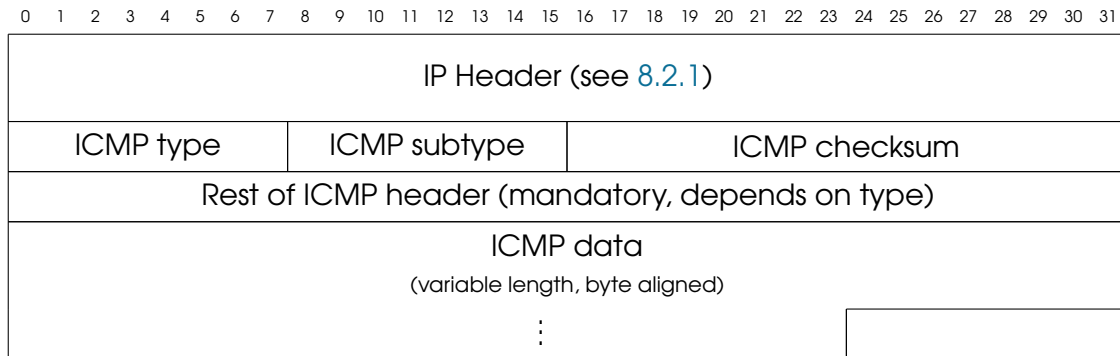
Exercise 8.24 What impact could it have to poison a victim's ARP cache, changing the gateway router's cached MAC to the attacker's MAC?

8.7 ICMP – Internet Control Message Protocol

The Internet Control Message Protocol (**ICMP**) is an auxiliary protocol (technically part of **IP**) that is used to test and debug (inter)network communications.

8.7.1 Packet format

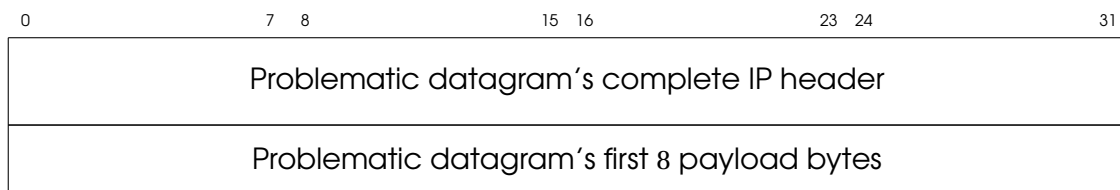
ICMP messages are encapsulated as the payload of IP datagrams. In the IP header, the **Payload protocol** is set to **0x01** (“ICMP”).



The most common ICMP type/subtype combinations you are likely to find nowadays:

ICMP type	ICMP subtype	Error?	Meaning
8	0		Echo Request ICMP message (ping).
0	0		Echo Reply ICMP message (ping reply).
3	0–16	✓	Destination Unreachable. Subtype explains cause.
11	0	✓	Time to Live Exceeded: TTL reached 0.

For these error ICMP messages, the ICMP data field contains information about the datagram that caused that error ICMP message:



For echo request and reply messages, the ICMP data field is arbitrary within the limits of IP datagrams. Fragmentation of ICMP datagrams (e.g., large ping payloads) is possible, but many routers will ignore ICMP fragments.

8.7.2 Operation

ICMP error messages are sent by **routers** and regular hosts, triggered by other datagrams:

- The “Destination unreachable” family of messages is sent when a datagram cannot be delivered to its destination, e.g., due to a misconfigured **routing table**, a closed **port** (see Section 9), or simply the destination host not being turned on.
- The “TTL exceeded” message is sent when a router receives a datagram for **indirect delivery** with **TTL** = 1 (see Section 8.3).

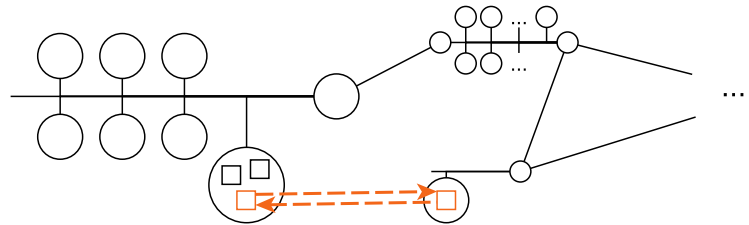
To prevent infinite amounts of traffic, ICMP errors are *not* produced when the offending datagrams are:

- ICMP datagrams (error or not).
- Datagram fragments, except for the first one.

Exercise 8.25 How can we differentiate the first datagram fragment from the following fragments?

9. Layer 4: Transport

Layer 7: Application
Layer 4: Transport
Layer 3: Internet
Layer 2: Network (LAN)
Layer 1: Physical



Capabilities

The **Transport** layer communicates **processes** (programs) running in potentially different **hosts**. These processes are identified using Layer 4 addresses called **ports**.

The transport layer offers two different ways of communication: without **connections** (**UDP**) and with connections (**TCP**).

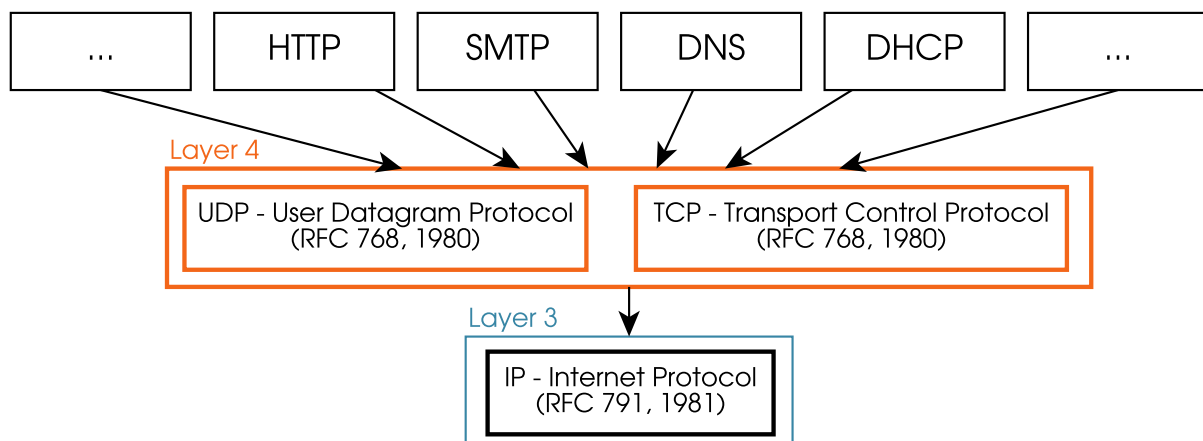
UDP doesn't establish a **connection** with the destination host because the priority is to send some data **payload** as fast as possible. Instead, UDP sends packets called **user datagrams** directly to the destination, without expecting any sort of confirmation of reception from the destination's UDP layer. UDP headers are simple and small as well keep the **overhead** of this protocol small.

TCP establishes a **connection** before sending any **payload** data to make sure both ends are ready. In addition, TCP expects **acknowledgement** (confirmation) of reception of every byte of payload data. Otherwise, data are **retransmitted** after a wait period, giving TCP **reliability**. TCP also uses a very clever way to limit the data exchange speed (**flow control**) so that fast and slow computers can coexist and communicate using TCP/IP. To provide all these features, TCP is a more complex protocol than UDP, uses larger headers and introduces a greater **overhead**.

Exercise 9.1 Which of the following applications do you think use **UDP** and **TCP**?

- Multiplayer gaming
- The WWW
- BitTorrent
- Video streaming

Protocols



The transport layer is at the top of the stack provided by the OS. Virtually all applications, and therefore their communication protocols, employ **TCP**, **UDP** (or even both) to exchange data over the net. Some important protocols that use TCP or UDP are shown in the previous figure.

Both TCP and UDP encapsulate their PDUs in **IP datagrams**. Generally, **IPv4** is used, but **IPv6** is also a possibility when available.



Neither TCP nor UDP change in any way when IPv6 is used instead of IPv4, *i.e.*, there is not a TCPv6 protocol. Instead, `tcp6` and other expressions that include TCP and 6 refer to (regular) TCP over IPv6.

9.1 Layer 4 addressing

TCP and UDP “addresses” are called **ports** and are 16-bit unsigned integers. To humans, ports are expressed in two main ways:

- As a number, *e.g.*, 22, 80 or 443.
- As the name of the service that typically uses that port, *e.g.*, **sshd**, **http**, **https**.

When a Layer 4 PDU is received, the port in its header is used by the OS to know which of the running processes must receive the data. In turn, processes have to **bind** to a TCP port or to a UDP port (or both) to let the OS know the port they expect data from.

Only one process may bind to a port at a time for sending data, but TCP and UDP ports are treated independently:

- If a process is bound to port **TCP/100**, it will not receive UDP traffic to port 100.
- If a process is bound to port **TCP/100**, another process (or even the same one) can bind to **UDP/100**.

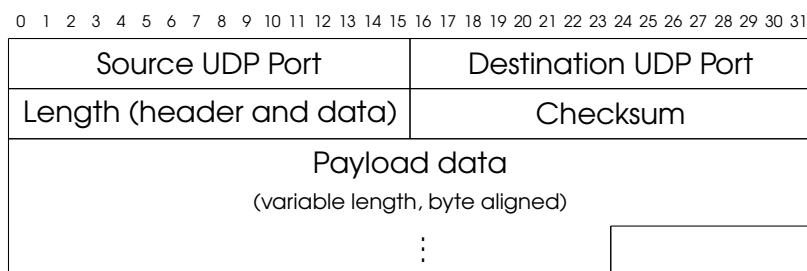
Port number **0** is reserved, so the usable range of ports is **1–65535**. Ports **1–1023** are only bound to processes with privileges (*e.g.*, an **http server**), whereas client applications (*e.g.*, an **http client**) can only bind to ports **1024–65535** based on their availability.

Exercise 9.2 How many different servers can be run in a machine to provide some feature to other hosts in the network?

9.2 UDP – User Datagram Protocol

9.2.1 Packet format

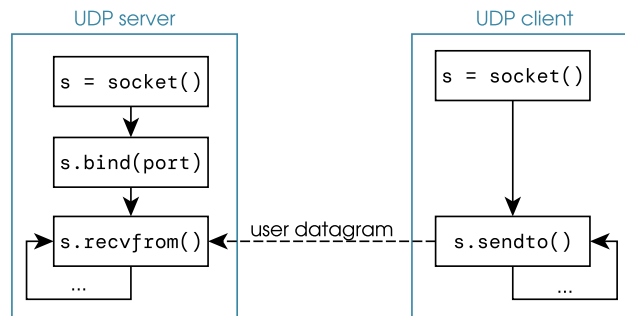
The **user datagrams** used by UDP all have an 8-byte header and have the following structure:



Exercise 9.3

- What is the maximum payload size that can be sent in a single **user datagram**?
- Is **fragmentation** desirable when using UDP?

9.2.2 Operation



UDP clients first create a **socket** and then directly send the user datagrams with `sendto`. There is no need to establish or close a connection.

UDP servers must first **bind** to the port to be exposed to clients, and then receive each datagram in an individual call to the read function (`recvfrom`).

The `bind` function is purely internal to the server: it entails changes in its configuration (the table mapping port numbers and **processes**), but it does not generate any message.

Exercise 9.4

- Can a UDP server work without sending any data through the network?
- Can a client receive UDP traffic without binding to a port?
- Can two different hosts bind to the same port and still exchange **user datagrams**?

Exercise 9.5 The following snippet of code shows a basic UDP server skeleton:

```

.py snippets/udpserver.py
1  import socket
2
3  port = 4567
4  s = socket.socket(family=socket.AF_INET, type=socket.SOCK_DGRAM)
5  s.bind('', port)
6  while True:
7      user_datagram = s.recvfrom(1024)
8      print(f"Received UDP on {port=}: {user_datagram!r}")
  
```

- Run the server, and in another terminal run `ncat -u localhost 4567` and type some text to test the server.
- Implement the client end of the communication and test it against the server.
- What important information do you get if you change `s.recvfrom` to `s.recvmsg`?

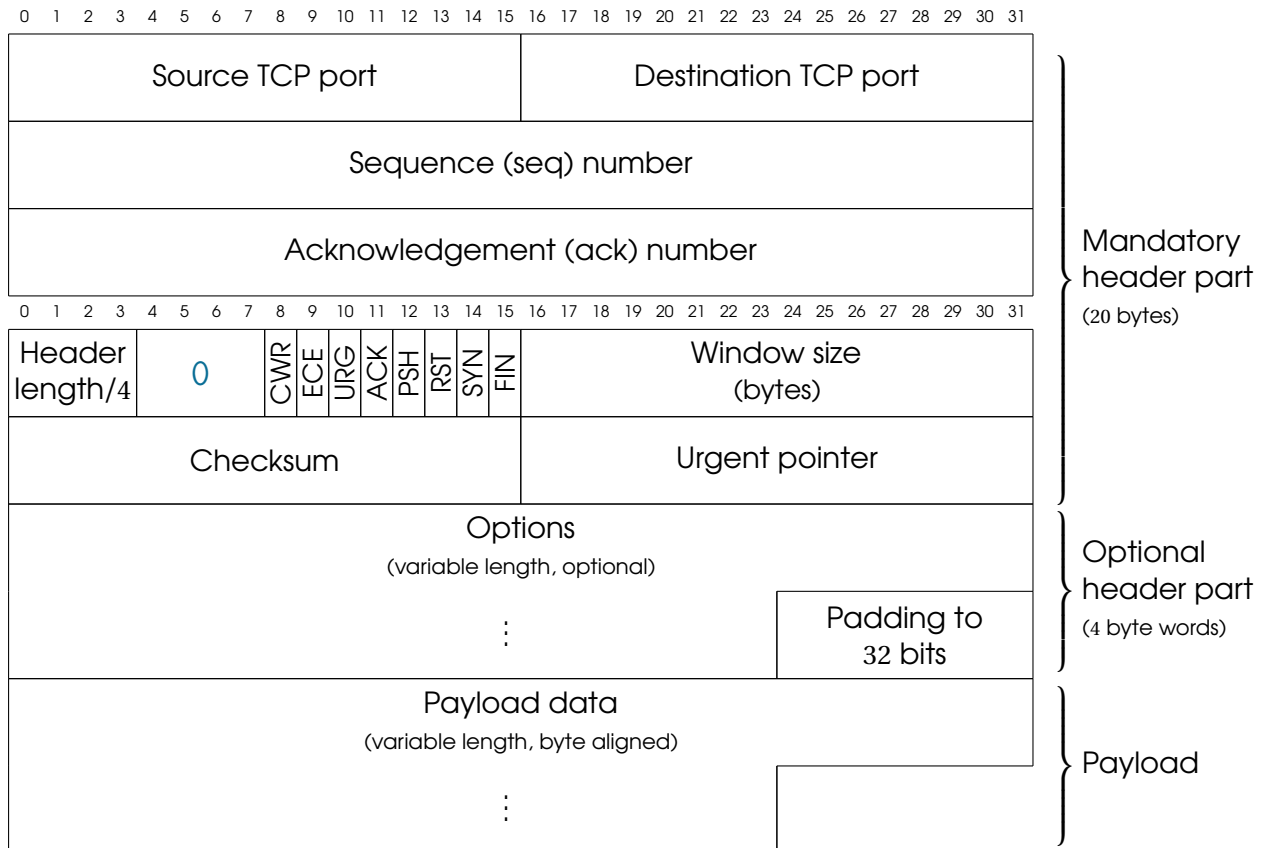


This code uses regular UDP **sockets**, so it does not need administrator nor the `CAP_NET_RAW` capability, as it happened in Exercise 7.4, which required access to Layer 2 (e.g., Ethernet) sockets instead.

9.3 TCP – Transport Control Protocol

9.3.1 Packet format

TCP PDUs are called **segments**. Their **header** is at least 20 bytes long, but can grow longer when certain options are used:



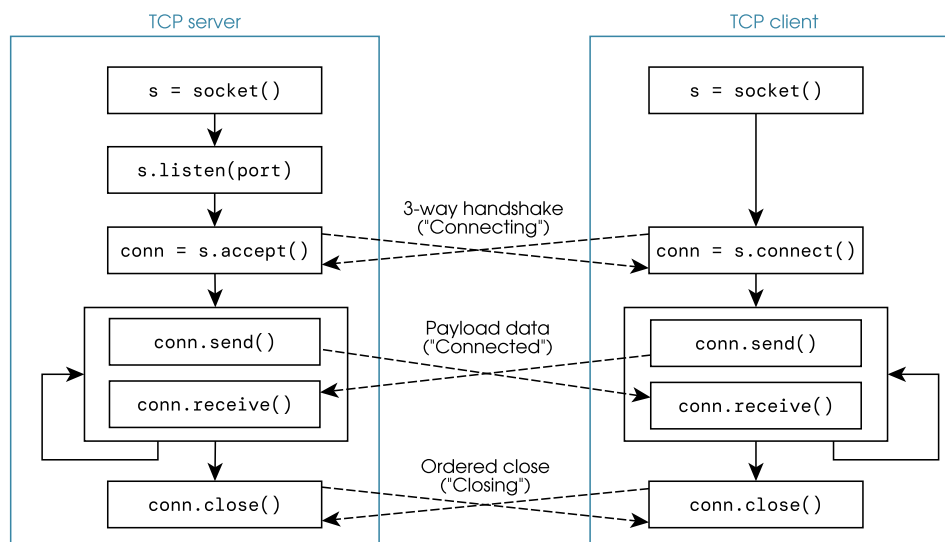
TCP headers contain eight 1-bit flags. Of these, four are used to core TCP features addressed in the following sections. In the order they appear:

- ACK: **acknowledgement** flag – “my ack number is valid”
- RST: **reset** flag – “*abort* the connection”
- SYN: **synchronization** flag – “I’m opening *my* end of the connection”
- FIN: **finalization** flag – “I won’t send any more *payload* data”

Exercise 9.6 How can the receiving end know the size of the options and of the payload data?

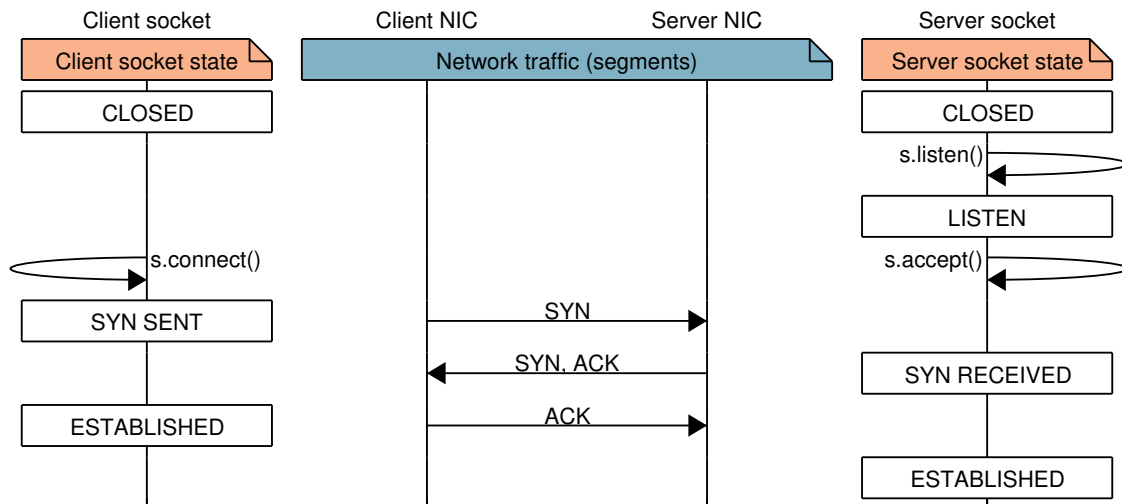
9.3.2 Operation

TCP is a **stateful** protocol. TCP **sockets** transition between multiple states during communication. Each end of the communication has its own socket with its own state. Roughly speaking, connections can be classified as “Connecting”, “Connection established”, or “Closing connection”. Payload data is only meant to be exchanged while in “Connection established”.



TCP sockets can be in one of 11 states. These states, together with two 16-bit counters (**acknowledgement number** and **sequence number**) enable the communication **reliability** expected of TCP. The meaning of these states and their transitions are explained in the following chapters.

9.4 TCP: “Connecting”



When a **socket** is created, it begins as **CLOSED**. The end acting as a server begins listening to the service's configured **port** and changes to the **LISTEN** state. Payload is not interchanged in this state.

A connection is established via a **3-way handshake** using the **SYN** and **ACK** header flags. Each end of the communication reaches the **ESTABLISHED** state at a different time thanks to this handshake. Once they do, they are ready to exchange payload data.



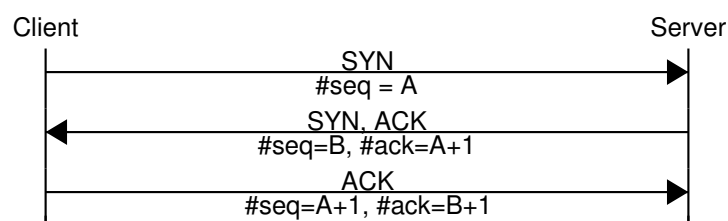
In TCP, both ends communicate as equal peers, it is unimportant who started the communication. That said, normally one end performs an “active open” (the client) and the other a “passive open” (the server).

Each end of the communication keeps track of its **acknowledgement number** (**#ack**) and **sequence number** (**#seq**) counters. The value of **#seq** is chosen separately (and typically randomly) by each end: $A, B \in [0, 2^{32} - 1]$.

Once during the connection opening, each end sends exactly one segment with the **SYN** flag enabled. These segments exchange (synchronize) their initial sequence numbers A and B .

Segments with the **ACK** flag enabled (all except for the client's first segment) have their **#ack** set to “the next **#seq** expected from the other end”. This **#ack** should be read as “I have correctly received all segments corresponding to, but not including, **#ack**”.

The expected **#ack** and **#seq** numbers during the **3-way handshake** are:



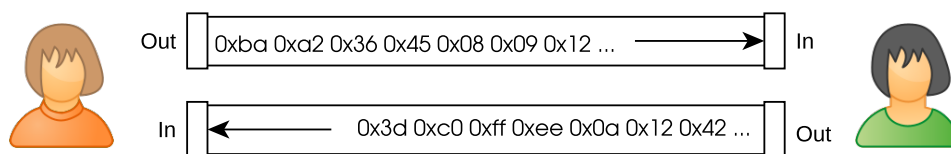
Exercise 9.7

- How can one end know that the other end has received their chosen **#seq**?
- Why does the third segment have its **#seq** set to $A + 1$ and not A ?
- What **#seq** will the next segment sent by the server have?

9.5 TCP: “Connection established”

9.5.1 Streams and sequence numbers

TCP offers the illusion of a data **stream**: two pipes of data in the In and Out directions (a **full-duplex** connection). These pipes are **reliable**, *i.e.*, mechanisms are put in place to prevent data loss, duplication and reordering.



Every byte of payload data is assigned a sequence number (**#seq**) in the TCP header. The first byte of payload data has **#seq** = $A + 1$, where A is the initial sequence number established during connection opening (see Section 9.4). Each new byte of data is assigned the next **#seq** (no gaps).

#seq	A	$A + 1$	$A + 2$	$A + 3$	$A + 4$	$A + 5$	$A + 6$	$A + 7$	$A + 8$	$A + 9$	...
content	SYN	0x48	0x69	0x21	0x0a	0x48	0x6f	0x77	0x20	0x61	...
		'H'	'i'	'!'	'\n'	'H'	'o'	'w'	' '	'a'	...

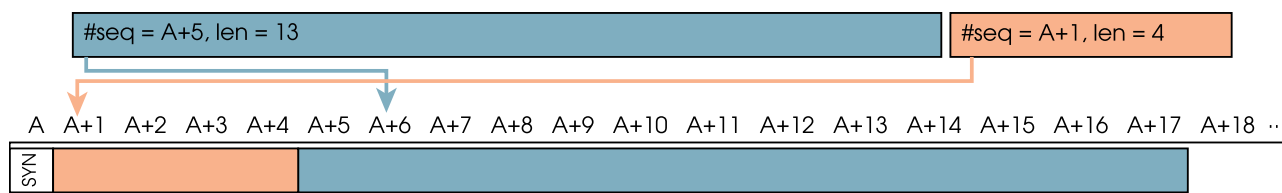
Exercise 9.8 The **#seq** that comes after $2^{32} - 1$ is 0. Why?

Payload data in TCP is always sent together with the *first* position they occupy in the **stream** and their length. For instance, inferring from the data shown above, two segments could be produced:

1. **#seq** = $A + 1$, **len** = 4, data = 0x48 69 21 0a ('Hi!\n')
2. **#seq** = $A + 5$, **len** = 13, data = 0x48 6f 77 20 61 72 65 20 79 6f 75 3f 0a

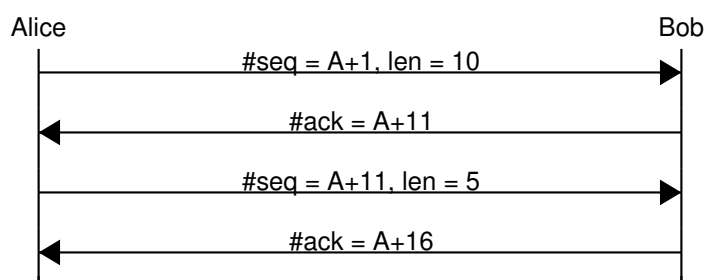
Exercise 9.9 What would be the **#seq** of the next segment with new payload data?

The receiving end of a TCP **segment** uses this information to stitch together a copy the data in the right order, even if the segments themselves arrive out of order. These data are passed on to the upper **layer** in a **buffered** fashion, or directly if the **PSH** (push) TCP flag is enabled.



9.5.2 Reliability and retransmission

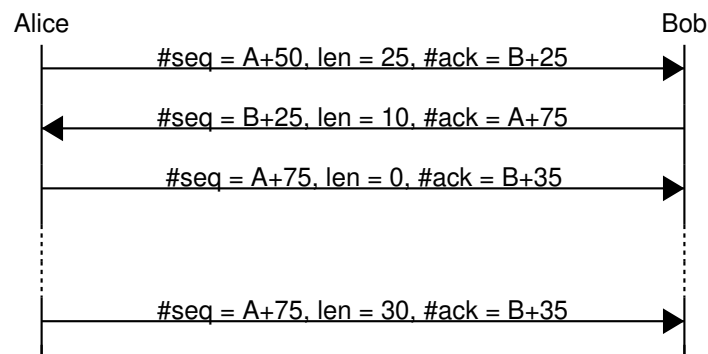
TCP needs all transmitted data be **acknowledged** by the other end. Whenever some data is received in a TCP segment, the receiving end sends back a confirmation even if no data need to be sent at that point. The acknowledgement number (**#ack**) in the TCP header is used to tell the other end that all data up to (but not including) **#ack**.



 The **#seq** field of a TCP header refers to the sender's sequence numbers. The **#ack** field refers to the receiver's sequence numbers.

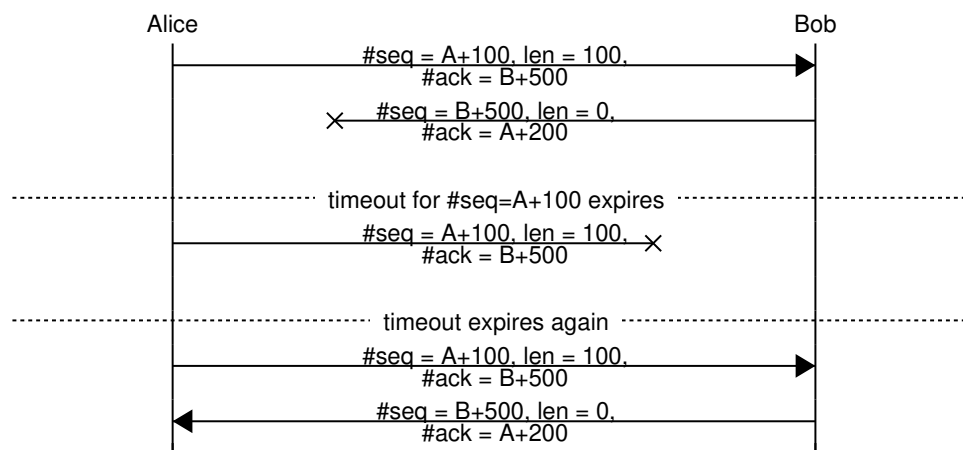
Payload data is included in TCP confirmations whenever any data is available. Conversely, the most recent value of #ack is included whenever any data is sent, *i.e.*, acknowledgments piggyback payload data.

For instance, if Alice sends a 25-byte request, Bob may send his 10-byte response and the acknowledgement for Alice's data in the same TCP segment. Alice needs to acknowledge Bob's data, and soon sends a segment with no data if needed. Alice will resume sending data later when needed.



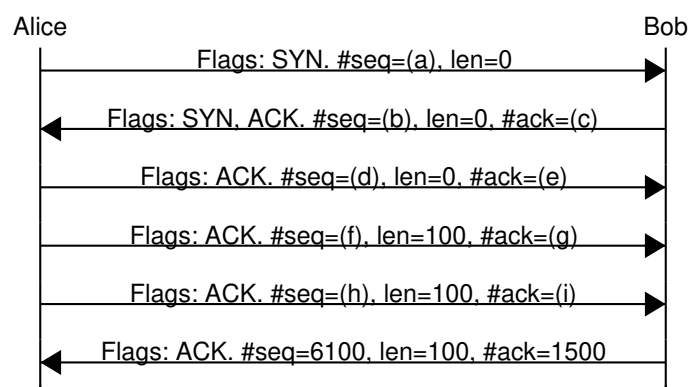
A TCP segment that is not acknowledged after a certain wait period is **retransmitted**. That time is decided by the OS on each end based on the average time it takes for a segment to be sent and a reply be received (the round-trip time, **RTT**).

Acknowledgements are sent even if that acknowledgement is identical to a previous one. This is because acknowledgements can also be lost.



Exercise 9.10 Assuming both ends are correctly following TCP:

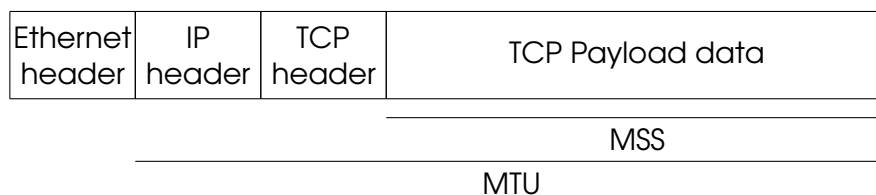
- Provide a set of valid values for (a) to (i).
- How does the story change if (f) and (h) are identical/different?



9.5.3 Transmission limits

Data for transmission is often produced in bursts. We would like to send as much as possible as soon as possible, but there are two limits put in place in TCP.

The first limit, the **Maximum Segment Size (MSS)** is the maximum amount of payload data per segment allowed in a TCP connection.



The reason for this limit is to prevent fragmentation, for performance reasons, e.g., due to the elevated cost in time and memory of reassembly.

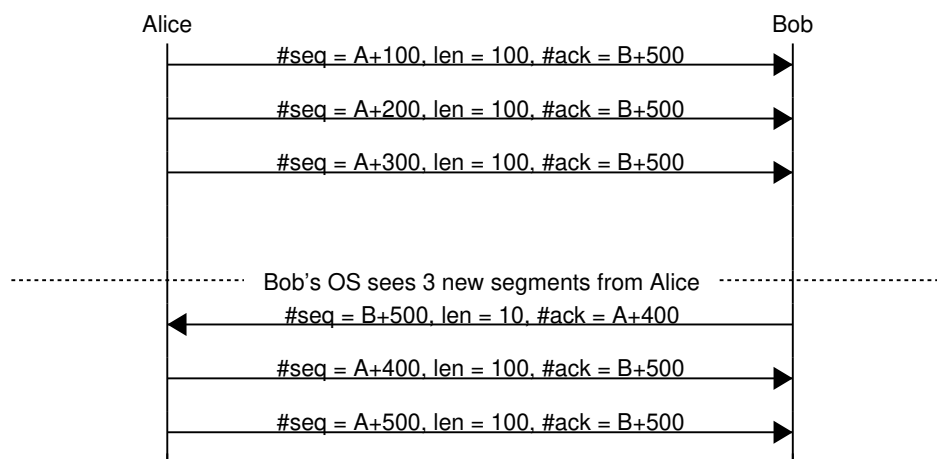
The MSS is 536 bytes in **IPv4** by default, but can be negotiated with the TCP header Options during the connection opening phase. In **IPv6**, the default MSS is 1220 bytes.

Exercise 9.11 Ethernet has an **MTU** of 1500 bytes. Assuming **datagrams** travel through 2 Ethernet networks only:

- What is the largest **MSS** that cannot cause fragmentation, $MSS_{\max}$?
- What is the **overhead** for $MSS_{\max}$ and for $MSS_{\max} + 1$?

The second limit is to the maximum amount of payload data that can be sent before the other end replies with a new **#ack**: the so-called **transmission window** size. A host will not send data (however much is available, or how urgent it is) until those **#ack** arrive. When the slower computer replies, it does so always with the most recent **#ack**.

Exercise 9.12 What is the most likely MSS and transmission window sizes for this example?



The value of the transmission window is announced with each and every TCP segment: this value is a mandatory field of TCP headers. As the connection progresses, each host can change the value they announce based on their own load and necessities.



Each host announces its own window size.

This limit is to prevent fast computers from overwhelming slower (or overworked) ones: without the limit the slower computer could need an infinitely increasing buffer size to store the not-yet-processed segments sent by the faster computer. With this limit in place, at the saturation point, both ends send segments at the same rate even if it takes vastly different times to produce and process them.

9.6 TCP: “Closing connection”

9.6.1 FIN, #seq and #ack

Once the connection is ESTABLISHED, it remains so until either end decides to close it.

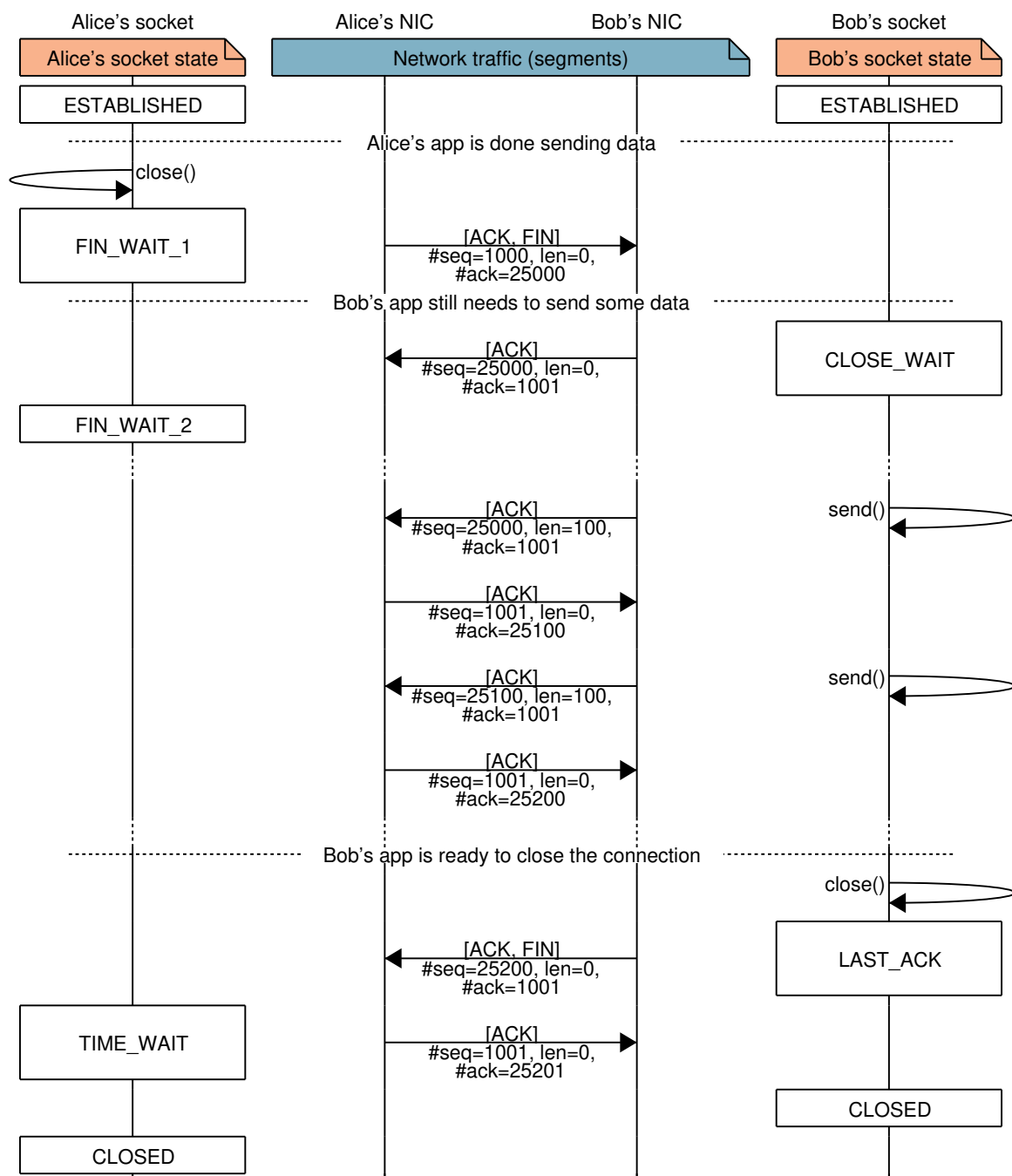
A node announces it is done sending data and wants to close the connection by sending a segment with the **FIN** flag active in the header. The other end may continue sending data, in which case it expects replies with updated #ack values. The other end may also choose to close the connection as soon as it sees the first FIN. Either way, if k bytes of data are sent and A is the initial #seq, then one segment with #ack = $A + k + 2$ is expected.

#seq	A	$A + 1$	$\dots$	$A + k$	$A + k + 1$
content	$\sum$	Data ...	$\dots$	... data	$\sum$

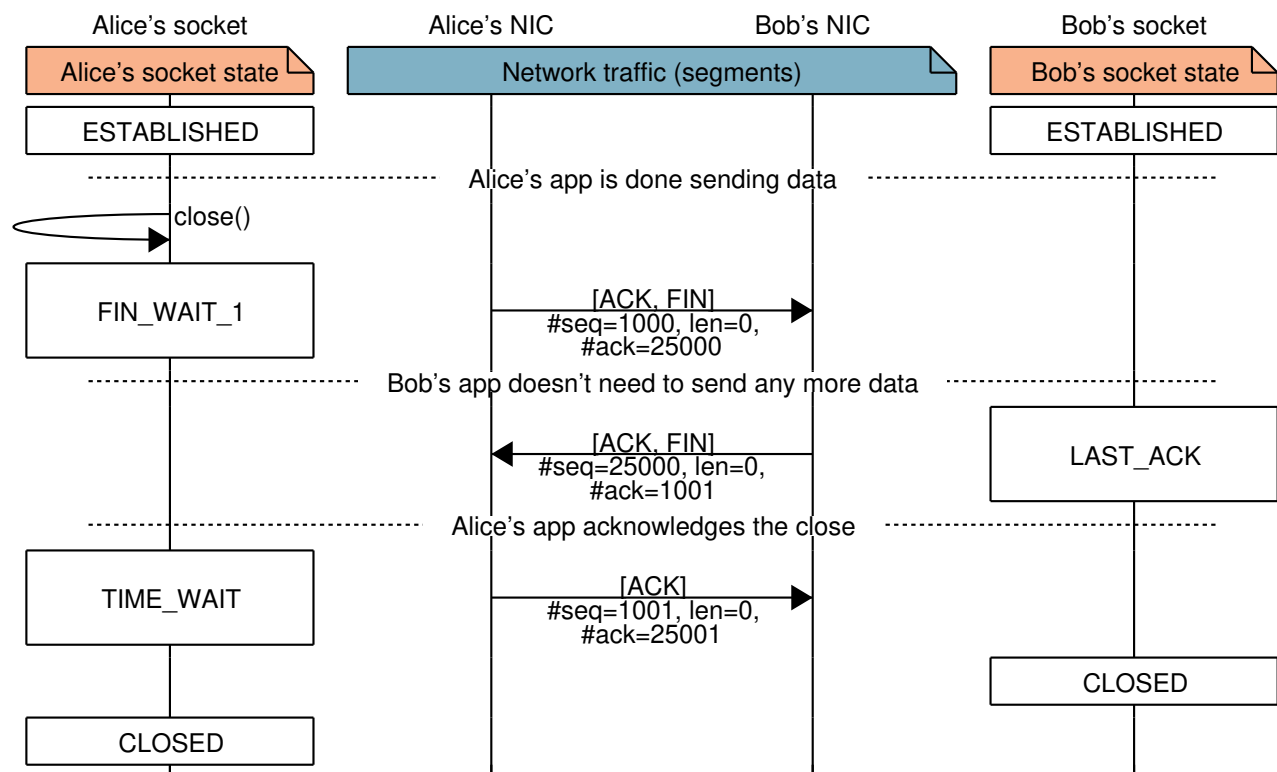
Note how FIN flag occupies a sequence number itself and must be acknowledged like the SYN (see Section 9.4), e.g.:

9.6.2 Two-sided close

After one end closes its side of the connection, the other may continue sending data indefinitely. When the second side is done sending data, it also sends a segment with the **FIN** flag enabled.



Note that, in case the other end wants to end the connection as it receives a FIN segment, the closing process is condensed in 3 instead of 4 segments, e.g.,



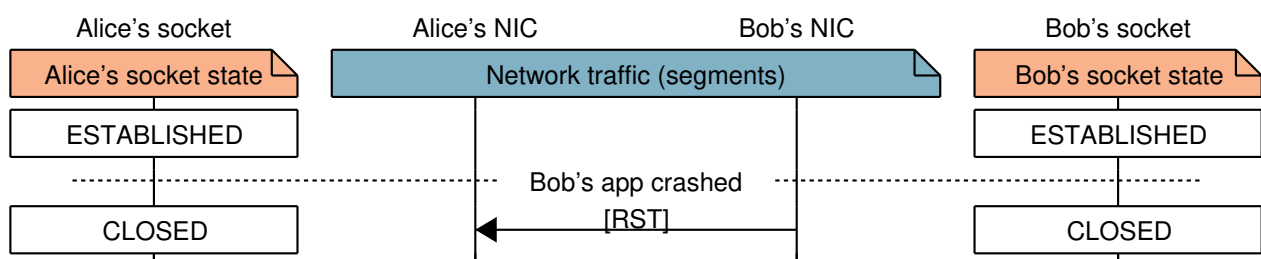
Note that:

- Each end sends only one segment with the FIN flag, except for retransmission.
- The segment with the FIN flag may contain payload data. In that case:
 - The payload data occupies sequence numbers *before* the FIN's.
 - The other end must ACK both the payload data and the FIN.

9.6.3 One-sided close

Sometimes, connections must be terminated and there is not enough time or interest to go through the multi-step, two-sided connection closing process. This may be the case, e.g., when a **process** crashes, a given TCP or UDP port is blocked, and other situations of error.

In this scenario, the host that detected the abnormal situation simply sends a segment with its **RST** (reset) flag enabled. The receiving ends should not acknowledge the RST; instead, the connection resources are directly freed, e.g.,

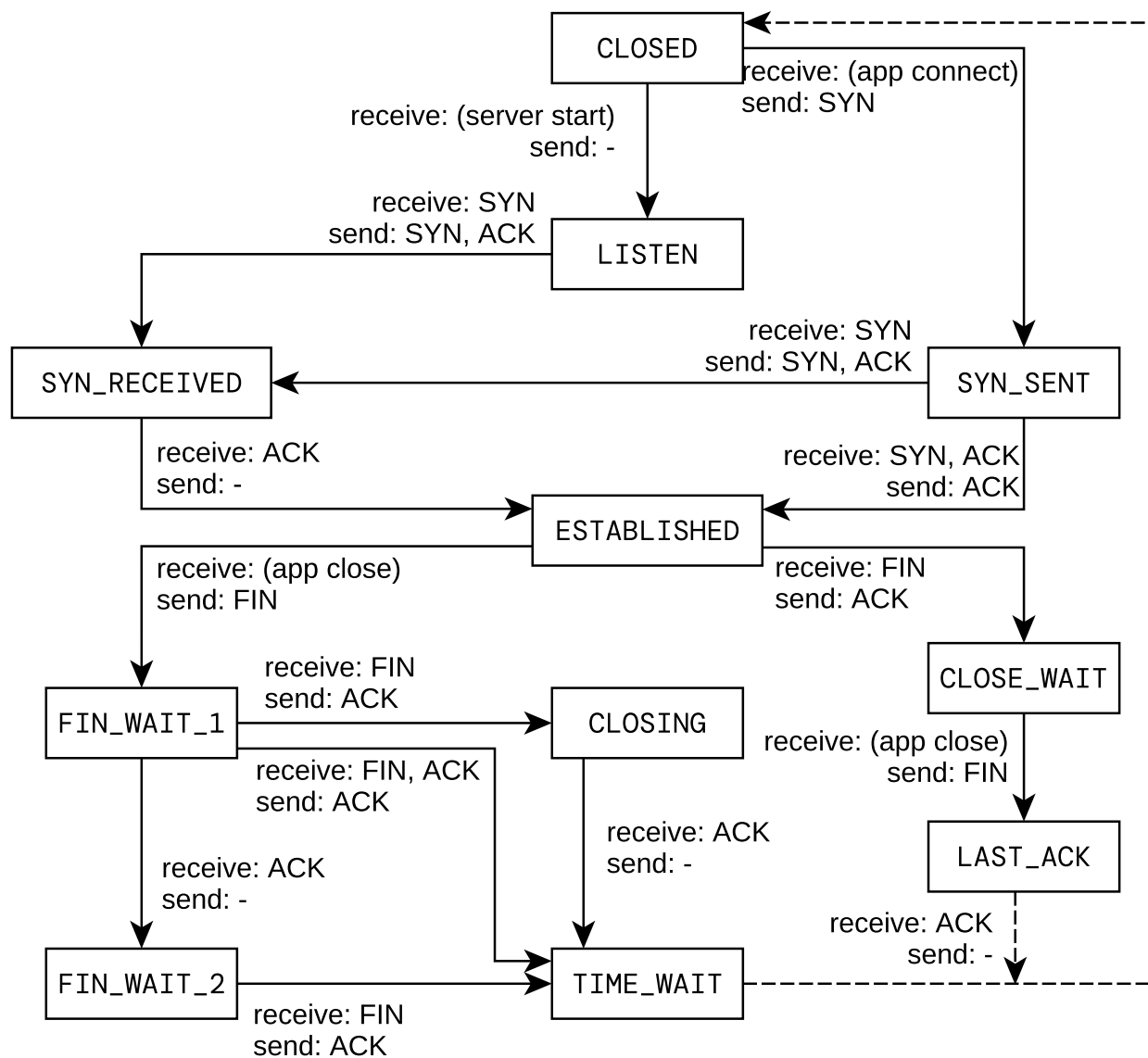


Following the two-sided or the one-sided connection close procedure helps reduce the computation resources required to maintain the network stacks of billions of devices worldwide.



A connection *cannot* be re-open after it is closed. The RST flag could have been more accurately called "forceful termination".

Exercise 9.13 The following diagram displays the 11 TCP states and (almost) all possible state transitions.



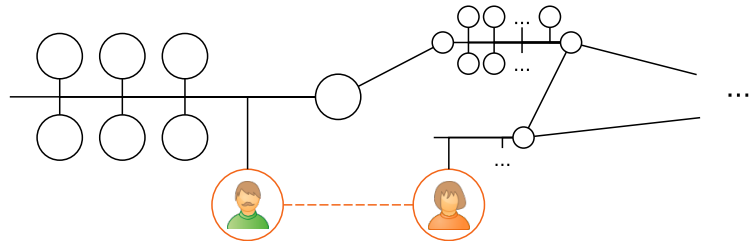
- Indicate the life cycle of an HTTP server's socket as a path in the graph.
- Indicate the life cycle of the corresponding HTTP client's socket as a path in the graph.
- The `CLOSING` state is not described in the previous sections. What do you think is its purpose?



This diagram is adapted from "TCP/IP Illustrated, Volume 2: The Implementation," by Gary R. Wright and W. Richard Stevens.

10. Layer 7: Application

Layer 7: Application
Layer 4: Transport
Layer 3: Internet
Layer 2: Network (LAN)
Layer 1: Physical



The top layer of the TCP/IP network stack includes all applications that use TCP or UDP for transport. It is by far the one that contains the largest number of protocols, and where those protocols are more quickly created and replaced.

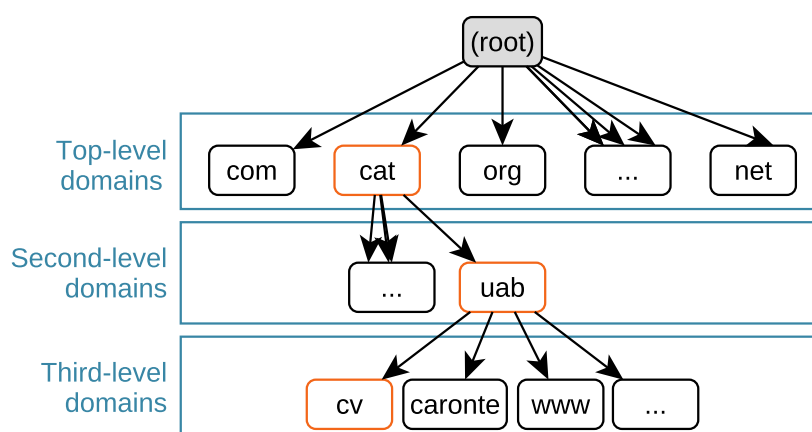
In addition to user apps, there are several auxiliary systems and protocols that help with configuration and operation of all hosts in the Internet. Some of the most critical are addressed in this chapter.

10.1 DNS – Domain Name System

The DNS protocol is responsible for translating domain names such as `ietf.org` or `cv.uab.cat` into IP addresses that can be placed inside datagrams.

DNS services are run on both TCP and UDP, i.e., both transport protocols can be enabled. In both cases, port 53 is used.

10.1.1 Hierarchy



Internet domain names are sequences of names separated by dots. These names are presented from the specific to the general in a hierarchical fashion. For instance, in `cv.uab.cat`, `cv` belongs to `uab.cat`, and `uab` to `cat`. The rightmost name, e.g., `cat` or `com` must be one of IANA's top-level domains.

DNS servers are organized in the same hierarchical way as those domain names. There are 13 root DNS servers (Root-A to Root-M), which are responsible for all top-level domains. You can ask any of them about `com`, but also about `su`, and they will know about them.

That does not mean that any root server knows about every domain under their top-level domain (e.g., `uab.cat`): they just know about those top-level domains. (e.g., `cat`). The job

of the root DNS servers is to redirect you to another DNS server in charge of the **top-level domain** you requested.

Similarly, the DNS servers responsible for **cat** will know about any existing ***.cat** domain (e.g., **uab.cat**), but not about subdomains (e.g., **cv.uab.cat**). The job of those top-level domain DNS servers is to redirect you to other DNS servers **authoritative** for the second-level domain requested (e.g., **uab.cat**). Second-level domains can then structure their name **zone** as desired, including any subdomain number and depth, as well as the name to IP translation *within* the zone of authority.

Exercise 10.1 Everyone uses DNS all the time. What would be the risks of having a centralized name translation system instead of DNS's distributed nature.

10.1.2 Records

DNS can be seen as a distributed database with different types of entries called **records**. The main ones are:

A: An **A** entry maps a domain name to an IPv4 address.

AAAA: An **AAAA** entry maps a domain name to an IPv6 address.

NS: An **NS** points to a DNS server that can help.

MX: A **MX** entry provides the domain name of the email server that handles messages for a domain.

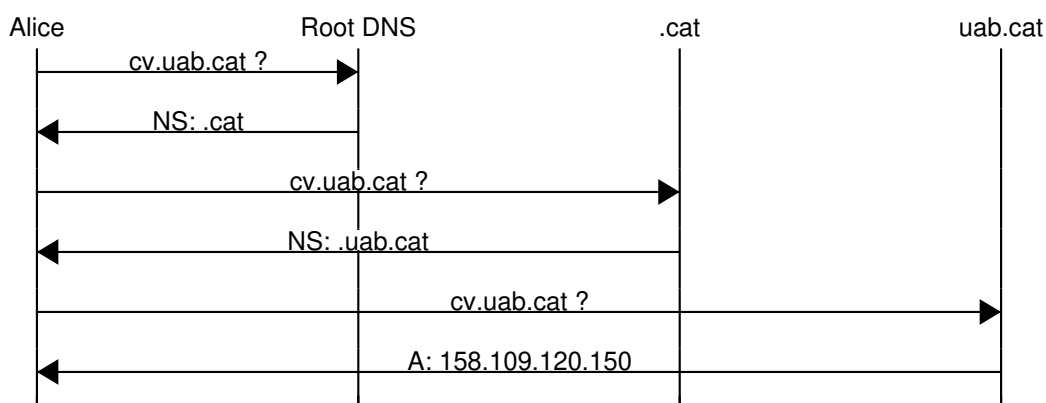
CNAME: A **CNAME** entry maps a domain name to another domain name.

Applications make **requests** to DNS servers indicating the type of records they are interested in. Server **replies** may be for the requested **A** or **MX** records, but the client may also receive a redirection to another DNS (**NS**) or an alias to another domain name (**CNAME**).

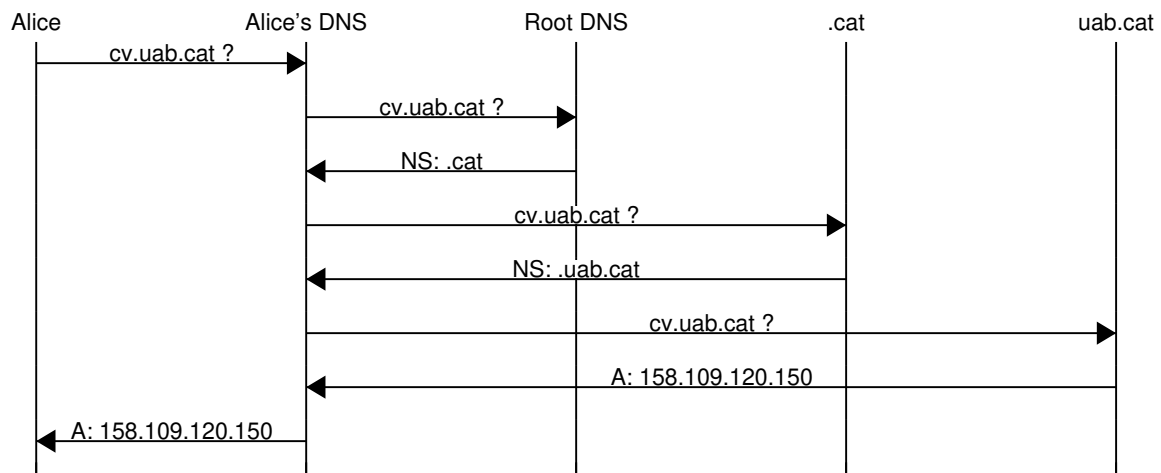
10.1.3 Resolution

The distributed and hierarchical nature of the DNS database makes it sometimes necessary to exchange multiple messages to satisfy a DNS **query**.

For instance, Alice may perform a fully **iterative** resolution all on her own. The first two replies contain an **NS** entry pointing to the next DNS server and the IP address of that DNS server, but not the IP address of the requested **cv.uab.cat**.



Alice's internet provider (**ISP**) or employer will likely offer DNS servers whose job is to receive Alice's requests and resolve them for her in a **recursive** manner.



The combination of **recursive** resolution and **cache** tables makes DNS a pretty efficient system with extremely high availability and low latency.

Exercise 10.2 You can use the `host` and `dig` tools to make DNS queries and see the results. For instance, my output to `dig uab.cat -t ns` contains the following lines:

```
;; QUESTION SECTION:
;uab.cat.                IN      NS
;; ANSWER SECTION:
uab.cat.                 172800  IN      NS      dns.uab.es.
;; ADDITIONAL SECTION:
dns.uab.es.              172112  IN      A          158.109.0.1
```

- What type of record did I request?
- What two record types did I receive?
- Why did I receive two record types?
- Perform a manual iterative resolution of `cv.uab.cat`. You may start with something like `dig @202.12.27.33 cv.uab.cat` to query the M root server.

10.2 DHCP – Dynamic Host Configuration Protocol

In order to fully participate in the Internet, a host needs to have the following aspects correctly configured:

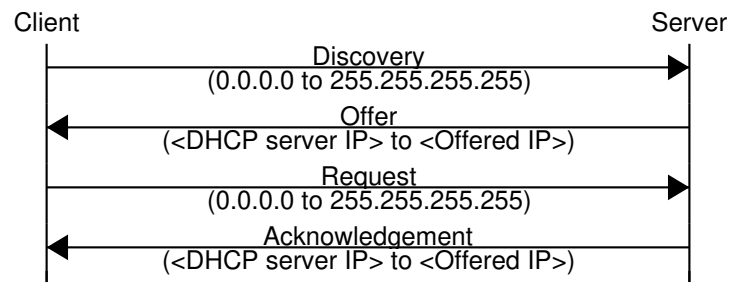
- **IP address:** The IP address is needed to identify a node in the network, and for that node to know what messages are directed to it.
- **Netmask:** Nodes need to know this to determine when to deliver datagrams directly, and when indirectly.
- **Gateway IP:** The IP address of at least one router is needed to deliver messages indirectly with destination outside the current LAN.
- **DNS server's IP address:** Although not strictly required to use TCP/IP, the address of a DNS server is extremely useful, and required to resolve domain names into IP addresses.

Traditionally, these parameters had to be manually configured in each computer. Nowadays, most user's computers are automatically configured with **DHCP**. Modern OSs contain a DHCP client that periodically queries the LAN until configured.



Network hardware comes with a pre-configured, default MAC address. Furthermore, a node needs to know its MAC address to operate the DHCP protocol. Therefore, MAC addresses are not configured via DHCP.

DHCP works on UDP. DHCP servers listen on port 67, and clients listen on port 68. The ideal DHCP exchange comprises 4 messages: Discovery, Offer, Request, Acknowledgement.



• Discovery

The goal of this message is to find DHCP servers willing to provide a configuration.

In the first communication, the client doesn't know the DHCP server's IP. Instead, it uses:

- Its MAC address as the source MAC address.
- The LAN's broadcast (`ff:ff:ff:ff:ff:ff`) as the destination MAC address.
- `0.0.0.0` as the source IP, meaning "unknown".
- The local broadcast address, `255.255.255.255`, as the destination IP.

• Offer

Zero or more servers will reply to the client with configuration offers. These configurations (e.g., IP addresses) are not property of the client until the server sends a DHCP Acknowledgement.

The server keeps track of a pool of addresses that dynamically assigns to clients. IP addresses are offered for a fixed period of time, after which they expire unless they are renewed (see below). Addresses that are not renewed are put back in the pool and offered back to clients as they make requests.

- The server can use its real IP address and the one offered to the client.
- The client knows it's the recipient because the server pays attention to the client's Discovery source MAC address.

• Request

The client picks one of the received offers and requests that the assignment is completed.

- The client uses the local broadcast address, `255.255.255.255`, as the destination.
- The client maintains `0.0.0.0` as the source because the offered address is still not the client's.

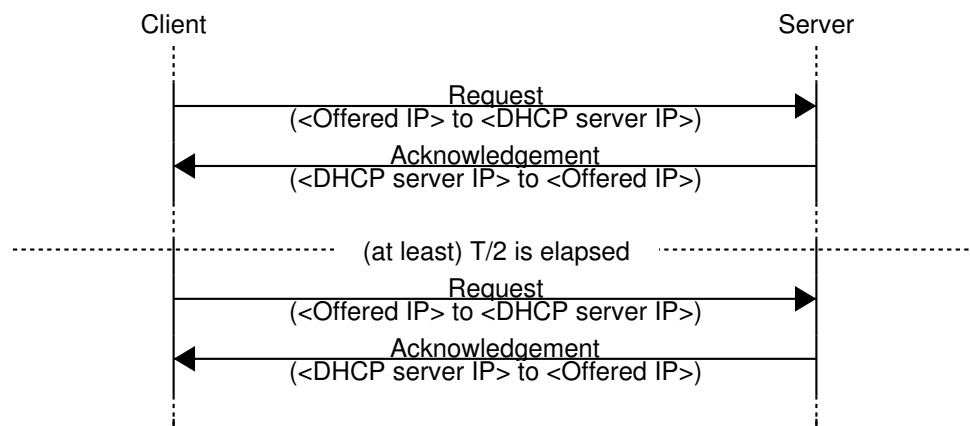
• Acknowledgement (ACK)

The server confirms the request. Once the client receives this ACK, it starts using the offered configuration as its own. This ACK should not be confused with TCP's.

• Negative Acknowledgement (NACK)

It is also possible that the server denies the request, e.g., if too much time has passed since the offer. In this case, a negative Acknowledgement (NACK) is sent instead. If a NACK is received, the client needs to go back to the Discover phase.

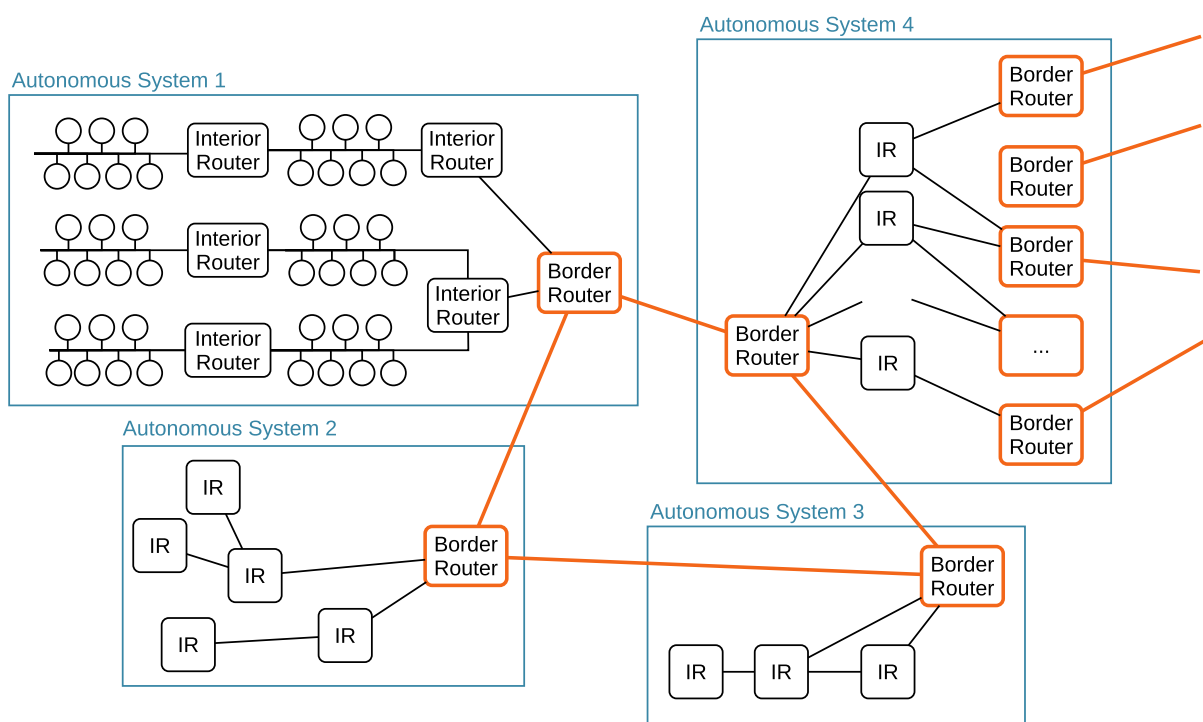
When a DHCP server leases an address to a client, it does so for a limited period of time T . This period is set by the DHCP server, and can be configured. After $T/2$ is elapsed, clients may try to renew the leased configuration. To do so, the Request-Acknowledgement cycle is repeated over time until either end fails to complete it in time, or the DHCP server sends a NACK.



Exercise 10.3 The typical domestic router implements both a DHCP client and a DHCP server. Explain how this is possible and why it is useful.

10.3 Autonomous Systems

Routing is performed one **hop** at a time, using **routing tables** as discussed in Section 8.3. Those tables need to be coordinated at a large scale to guarantee graph connectivity to all hosts in the Internet.



10.3.1 Organization

To facilitate this coordination, LANs are grouped in **autonomous systems (AS)**. These manage non-overlapping blocks of IP addresses (**w.x.y.z/N**) that are separately owned and managed. Many companies, universities and ISPs have their own ASs that are publicly **listed**. For instance, UAB's network belongs to AS13041.

Internet Exchange Points (IXP) are a special kind of AS whose purpose is to interconnect other ASs, instead of hosting services and/or clients. Of course, running these IXPs is not free, so commercial relations are established with those interconnected ASs. Some IXPs are committed to providing service to anyone meeting their basic requirements, are not controlled by any single operator, and offer transparent and often equitable pricing to their clients. This type of IXP is called **Neutral Point (NP)**, as opposed to nonneutral points used, e.g., only by a single network operator.

All ASs operate **border routers (BR)** to interconnect ASs. When a border router receives a datagram with destination inside the AS, it forwards the datagram through the AS' **internal routers (IR)**. Similarly, when a host inside an AS needs to contact a host in another AS, datagrams are forwarded through border routers until they reach the destination AS and ultimately the destination LAN.

10.3.2 Routing table configuration

Manually configuring the **routing tables** of all border routers and internal routers would be too costly and prone to errors. Instead, different algorithms are employed to configure them automatically.

- Within the AS, internal routers and border routers use one of the available **interior gateway protocols**. Often, **RIP** (Routing Information Protocol) is used due to its relatively simple configuration. That said, ASs may freely choose the algorithm they use internally to their AS.

RIP implements a variation of the Bellman-Ford shortest path algorithm to minimize the number of **hops** datagrams need to travel within the AS. Periodically (e.g., every 30 s), each IR and BR:

1. Advertises its *perceived* distance to other routers in the AS.
2. Updates its perceived distances based on other router's advertisements.
3. Updates its **routing table** to all LANs in the AS accordingly.

Exercise 10.4 RIP uses the number of hops as its cost metric. How could this be problematic? How can this problem be solved?

- Between ASs, only border routers need be configured; interior routers simply need to direct exterior traffic through the border routers.

Border routers run an **exterior gateway protocol** in addition to the interior gateway protocol used within the AS. Different ASs must agree on the algorithm they use, so **BGP** (border gateway protocol) is normally used.

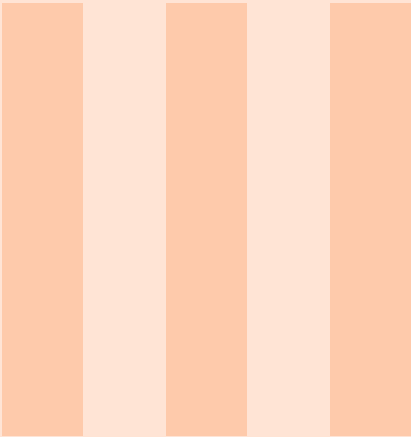
BGP is a more complex protocol than **RIP**. This complexity is needed because BGP coordinates border routers belonging to different ASs, *i.e.*, with different owners:

- RIP uses a single number (the distance, *i.e.*, the number of **hops**) to guide a shortest-path graph algorithm. BGP uses **paths** instead of distances. This allows BGP to choose routes with more **hops**, but more desirable to the AS, *e.g.*, in terms of time, monetary cost, internal load, etc.
- BGP was designed with the notion of security and trust in mind, and includes mechanisms to verify the authenticity of the paths announced by a border route.



BGP can also be used as the AS' interior gateway protocol. This is typically done only by very large ASs.

Exercise 10.5 How can BGP translate economic interest into routing configurations?



Missions

The last part of this guide proposes 9 complex problems called missions with a retrofuturistic sci-fi flavor. These problems vertebrate the contents of the previous parts with a focus on the applications and the goal of reaching deep concept understanding.

Students are encouraged to work all these missions in order, and to produce solutions in the shape of formal reports. They are also encouraged to shared them with their immediate learning community (e.g., their class) and to exchange feedback with formal letters.

11	Missions	63
11.1	Mission 1: Sharing is caring	
11.2	Mission 2: Infinite scroll	
11.3	Mission 3: Herding chaos	
11.4	Mission 4: Inter { planetary net	
11.5	Mission 5: Money is the root	
11.6	Mission 6: Annoy two whole planets	
11.7	Mission 7: To TCP or not to TCP	
11.8	Mission 8: You talking to me?	
11.9	Mission 9: Interplanetary Hallway	

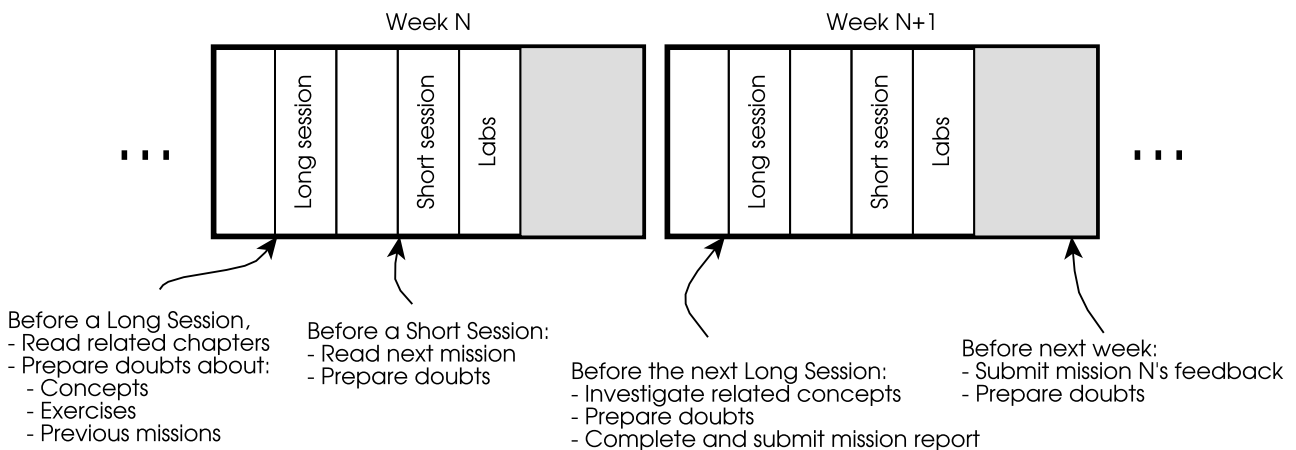
11. Missions

Mission contents

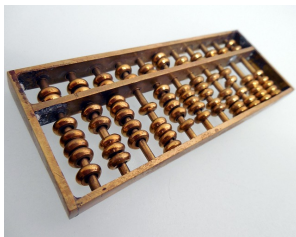
		Missions								
Chapters		M1	M2	M3	M4!	M5!	M6	M7!!	M8	M9
§2	Piercing the analog-digital veil	✓	🔄							
§3	Addressing and LAN Topologies	✓	🔄	🔄	🔄	🔄				🔄
§4	Packet format and MTU	✓	✓	🔄	🔄	✓	✓	✓	🔄	
§5	Network stacks and encapsulation	↔	✓	🔄	🔄	🔄	✓	🔄		
§6	The Internet: TCP/IP Overview	↔	↔	↔	✓	🔄	🔄	🔄	🔄	🔄
§7	Layer 2: LAN Communication		✓	🔄	✓	✓		🔄	🔄	
§8.1	Layer 3 addressing		↔	✓	✓		🔄			🔄
§8.2	IP function and header		↔	✓	🔄	✓	🔄			
§8.3	IP routing				✓	🔄		🔄		✓
§8.4	IP subnetting			✓	✓					✓
§8.5	IP fragmentation		↔			✓		🔄		
§8.6	ARP				↔	✓		🔄	🔄	
§8.7	ICMP				↔				✓	
§9.1	Layer 4 addressing						✓		✓	
§9.2	UDP							✓	🔄	
§9.3	TCP						↔	✓	🔄	
§9.4	TCP connection opening						✓	✓	🔄	
§9.5	TCP data exchange						↔	✓	🔄	
§9.6	TCP connection closing							↔	✓	
§10.1	DNS								✓	
§10.2	DHCP									✓
§10.3	ASs, IXPs, NPs									✓

✓: main focus — ○: review — ↔: foreshadow — !: critical

Suggested rythm



11.1 Mission 1: Sharing is caring



About 75 years ago, a top-secret expedition was sent through space with the goal of colonizing another planet. The expedition arrived safely at their destination, but the landing broke their communication instruments, leaving them stranded. You are a third-generation colonist, engineer-to-be, and this is the first mission assigned to you by the assembly: share the new and shiny calculator. **Please read everything carefully.**



Important: Each student is encouraged to post a **formal report** (in PDF) in the “Mission 1 exchange forum” of the Campus Virtual before the next 2h session. After that, we count on you to post a **formal reply letter with feedback** (also in PDF) for at least one other solution, before the following 1h session.

Report details

Right now, the colony consists of 3 separate bases (3 buildings). These bases have 3, 200, and 5000 electronic devices, respectively. The base with 3 devices has been built specifically to house a new, powerful and power-hungry calculator (it is one of the 3 devices). Everyone in the colony wants to have access to the calculator, but none of the devices have network capabilities *yet*. Your goal is to propose a convincing networking design that connects and satisfies everyone.

Prepare a brief report with your design, proposing clear and complete solution proposals to the following points:

1. How is the **analog-digital barrier** crossed to allow digital communication? Be concrete in the technologies you propose.
2. How are the devices physically connected? Provide a diagram with the exact **topology**.
3. What **addressing system** will be used to uniquely identify each device? Give two address examples, one of them being the new calculator's.
4. What **packet format** will the devices employ? Provide a concrete format diagram and the bytes of a full, example packet.

Feedback details

After posting your solution to the forum, find another one and **claim it** by replying to it with a short “I will provide feedback” message (one feedback letter per solution).

Then prepare a formal letter in PDF and post it to the same thread once it's ready (you can find an example feedback letter in the Campus Virtual). In this letter, include your feedback (both positive and negative), explaining anything you deem relevant or think it can help that solution. Make sure to touch on the following points:

1. Do you see any major flaw or incomplete point that would prevent the proposal from becoming reality?
2. If you did your report again, would you adopt any of the proposals of this solution?
3. Could you interpret the contents of the message included in the report? Include the bytes of a possible response, based on your understanding of that report.

11.2 Mission 2: Infinite scroll



After reviewing the first round of reports, the colony assembly is excited to explore this new *computer network* idea. However, before investing in any of the proposals, the assembly is requesting more complete and concrete designs. Specifically, you'll need to add the possibility of sending arbitrarily long, potentially infinite messages, and pay attention to the network stack structure (details below).

Each one of you is expected to post a **formal report** (in PDF) in the "Mission 2 exchange forum" of the Campus Virtual before the next 2h session with a new mission programmed (see the calendar in the Campus Virtual). After that, we count on you to post a **formal reply letter with feedback** (also in PDF) for at least one other solution, before the following 1h session.

Report details

The new report should be standalone so that your reviewer does not need to read your first one. The proposal you describe in this report does not need to follow your first one exactly: you may borrow as many ideas and introduce as many changes as you see fit.

1. Summarize the **physical layer and topological layout** of your computer networks (addressed in points 1 and 2 of the previous mission).
2. Indicate your network's MTU and **why it is necessary to define a finite MTU** value. With this restriction, explain **how you'll allow sending messages of arbitrary (infinite) length**.
3. Describe the (updated) **addressing system** and its limitations. You can keep or change the system you proposed in the previous mission as long as you ensure it is valid.
4. Propose a **3-layer network stack** (let's call them L3, L2 and L1, from top to bottom) in which the arbitrary message length feature is addressed exclusively in L3, addressing exclusively in L2, and L1 focuses only on the digital/analog and analog/digital conversions.
5. Include an example in which a message of length 1.5 times the MTU is sent. In your example, provide the **bytes input to and output from each layer** of the origin and destination of the communication.

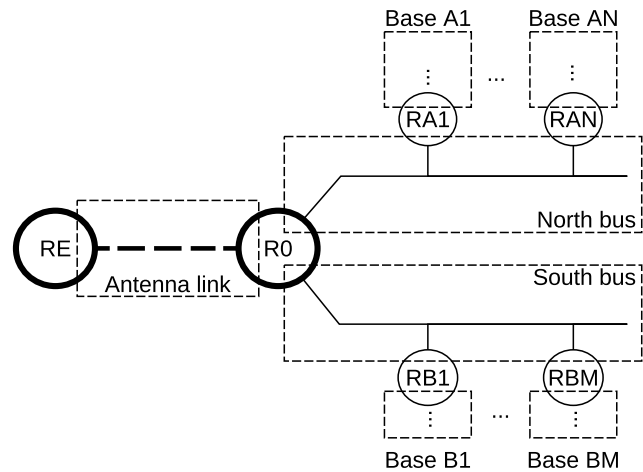
Feedback details

Use a similar feedback process as in the previous missions, making sure to touch on:

1. Do you see any major flaw or incomplete point that would prevent the proposal from becoming reality?
2. Is there any actual limit to the message length of the proposal you are reviewing?
3. Could you interpret the contents of the message included in the report? Include the bytes of a possible response of the same length, based on your understanding of that report.
4. Is there any part in the proposal you review that improves upon yours?

11.3 Mission 3: Herding chaos

Due to political differences, and in spite of the colony assembly's best efforts, each base in the planet adopted a different LAN technology (!). Even though communications are satisfactory within each base, at the moment it is impossible to directly connect any two computers from two different bases. Once again, the colony is asking for your help.



Thanks to everyone's sacrifices, the colony is now in a position to build and install a new router and two cabled buses. These are represented with a thicker line in the diagram below. Your long-term goal is to make it possible to have internetwork communications between any two nodes of any two bases. For now, you will be designing the physical and logical infrastructure.

Report details

All the bases employ the IPv4 protocol, and there are multiple bases connected to both the North and the South buses. Each base contains a single LAN. Coordinate with others so that *no two LANs have the same size*.

1. Within your group:

- Determine how many addresses you need so that each device can uniquely identified within your LAN. Choose the minimum LAN size that can satisfy this requirement, and quantify the free "space" available in your LAN.
- Indicate how many of the 32 bits of an IPv4 address you need to identify a single device connected to your base LAN. What concept used in IP is most closely related to this number? How can this structure be useful?

Network ID	Host ID
------------	---------

2. In coordination with the other groups in the classroom:

- Create a LAN for the North Bus, and describe all physical and logical addresses connected to it.
- Ensure that the IPv4 address blocks assigned to the LANs are of different size and do not overlap.
- Using the blackboard or a shared document, list all LANs, their sizes and (updated) address ranges.

Feedback details

Use a similar feedback process as in the previous missions, making sure to touch on:

1. Are the LANs correctly dimensioned?
2. Is there any collision between the blocks?
3. Did the original report extract all information about the IP address structure?

11.4 Mission 4: Inter { planetary net

After a few years of use, the colony is finally coming to appreciate the value of interconnecting heterogeneous networks using the IP protocol. So much that they have invested in a powerful HugeAntenna™ that, they think, can connect the colony with Earth's Internet.



The colony assembly is worried because they don't know the bases could communicate with each other and, more importantly, with the new antenna. Furthermore, they are afraid of Earth's despotic backlash if they don't manage to establish a connection with the Internet. Once again, they are asking for your help.

⚠ Warning! This mission is critical. Take your time, request as much help as needed, and keep your spirits up. Previous missions can be regarded as training for this one – it is strongly recommended you carefully finish them before attempting this one.

Report details

Recall the network topology requested in the previous mission. By now, you should have access to a consensus agreement on the physical and logical address distribution of the devices of the colony (otherwise, say something!).

Consider the *routers* present in the colony: $\{RA_i\}_{i=1}^N$ are connected to the North Bus, $\{RB_i\}_{i=1}^N$ to the South Bus, and R_0 links both buses. The new antenna will be connected to a third Network Interface Controller (NIC) of R_0 .

Your goal is to help configure all machines in the colony (especially the routers) so that information can flow between bases and, critically, between the colony and Earth's internet.

1. Summarize the physical and logical addresses of all machines in the colony so that your report can be standalone. It is suggested you reuse the configuration you produced in the previous mission after fixing any detected issue. Use the CIDR format whenever applicable.
2. Provide the simplest working routing tables you can conceive for the following machines:
 - One router RA_i of your choice, and a non-router machine M_i connected to RA_i .
 - One router RB_j of your choice, and a non-router machine M_j connected to RB_j .
 - The router R_0 .
3. Create two swimlane diagrams that show *all* packets transferred within the colony for the two following exchanges. For each packet in these diagrams, indicate *all* addresses contained in the packet. Be sure to specify somewhere the MTU of all LANs involved in the exchanges.
 - M_i sends some data to M_j , and M_j sends back a response.
 - M_i sends a message to a server on Earth, but receives no response.

Feedback details

Use a similar feedback process as in the previous missions, making sure to touch on:

- Is there any issue with the physical and logical address assignments?
- Was the CIDR format correctly employed?
- Do the routing tables contain all necessary fields?
- Can any of the tables be further simplified?
- Are direct and indirect deliveries properly depicted in the swimlane diagrams?
- Are the MTUs consistent with the diagrams?

11.5 Mission 5: Money is the root

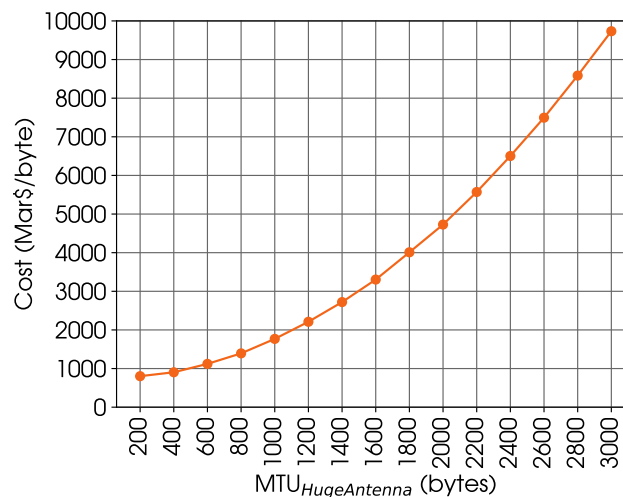
The whole colony is buzzing with intellectual energy after the installation of the HugeAntenna™ and its connection to Earth's Internet – think of all the cool stuff they have been missing all this time! Unfortunately, now that the colony is hooked on the Internet, Earth's tyrannical government has decided to greatly raise the cost of using the interplanetary antenna link. The abusive prices are causing great trouble in the colony's society, proving once more that money – or, perhaps, the lack thereof – can be the root of all evil. Will you help the colony regain financial stability?



⚠ Warning! This mission is critical. Take your time, request as much help as needed, and keep your spirits up. Previous missions can be regarded as training for this one – it is strongly recommended you carefully finish them before attempting this one.

Report details

Earth is willing to negotiate the price in Mar\$ (Martian Dollarroos) per byte transmitted by the HugeAntenna™. This price will depend on the MTU of the antenna as shown in the figure below. Note that $MTU \in \{200, 400, \dots, 3000\}$. The assembly needs a quantitative analysis that justifies the best choice of MTU.



To simplify your analysis, consider a representative scenario in which a user in one of the colony bases sends *payloads* of exactly 1900 bytes directly over IP.

1. Provide a topological map of the colony's network up to, but not including, the link between the HugeAntenna™ and the Earth. In those LANs, indicate the smallest MTU that prevents fragmentation within the colony. You may reuse the topology from previous missions or create a new one.
2. Consider at least 4 possible values for the HugeAntenna's MTU, 2 of them smaller than 1900 bytes and two of them larger. For each of these $MTU_{HugeAntenna}$ values, indicate:
 - How many packets are sent through the Huge Antenna to transmit the payload.
 - Which packet fields are not identical in those packets (only applicable if more than one packet is sent).
 - The total cost of sending the payload.

Coordinate with the other engineers to find the overall optimal $MTU_{HugeAntenna}$ value.

3. Compare the total traffic (in bytes) transmitted within the colony when the first payload is sent (and all caches are empty), versus the second payload (when caches are filled and valid).

Feedback details

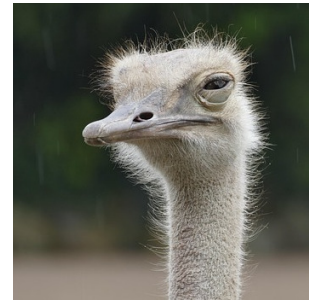
Use a similar feedback process as in the previous missions, making sure to touch on:

- Are the MTUs within the colony configured as requested?
- Is fragmentation correctly considered when determining the cost?
- Is the overhead due to ARP properly accounted for when calculating the total traffic?

11.6 Mission 6: Annoy two whole planets

Years have passed in the colony since you helped configure the large antenna that lets you communicate with Earth. Recently, your colony has found out about *another* secret colony, this one on Saturn – and Earth didn't even bother to tell us!

The assembly has decided that a little prank is in order, and is asking for your help once again. Your goal is to hijack the next Saturn ↔ Earth TCP connection and then send a forged memo impersonating Saturn. The assembly has requested that the memo be kept classy.



Report details

The colony's Chief Engineer has done some scouting work and is willing to help in your endeavor. She has gathered the following pieces of information:

- Your HugeAntenna™ on the colony communicates with another HugeAntenna™ on Earth. Earth uses that same antenna to communicate with another HugeAntenna™ on Saturn. When Earth receives the first packet of a TCP connection, they verify its origin.
- After that first packet, Earth doesn't verify the origin of any packet. In fact, bogus or unexpected messages received after that one will be simply ignored, without triggering any alarm. With careful timing, you will use this to inject your forged packets.
- The first day of each month at exactly 4:20 AM (Earth's New Barcelona time), Earth receives the first packet of a TCP connection from Saturn. These connections are used to exchange official memos, and the assembly wants you to hijack the next one.
- Saturn uses an outdated, insecure TCP/IP stack with the following flaw: it always sets TCP's sequence number to 0 in the first message of a TCP connection.
- You won't be able to see the messages exchanged between Earth and Saturn. However, the Chief Engineer was able to capture the following Ethernet frame, which initiated the previous TCP connection (let's call it C_{-1}) between Saturn and Earth (00 d0 is followed by f1 bb and 45 10 is followed by 00 28):

```
00 09 bf 42 07 64 00 d0    f1 bb 0f 5a 08 00 45 10
00 28 d2 71 40 00 40 06    00 00 ca f7 44 05 4c 4c
15 15 9f ee 00 a9 00 00    00 00 00 00 00 00 50 02
01 00 00 00 00 00
```

With this in mind, the assembly has requested a report that contains the following aspects:

1. Draw a swimlane diagram depicting the establishment of the previous connection (C_{-1}) between Saturn and Earth. Also, tabulate *all* header fields present in these packets (and their values).
2. Explain your attack plan, *i.e.*, how you plan to hijack the connection. Indicate clearly:
 - What assumptions you are making about the next connection (e.g., what fields will remain identical or could change).
 - How many messages you will be sending, and their planned timing.
3. Show the full sequence of bytes of a frame that contains your prank memo.

Feedback details

Use a similar feedback process as in the previous missions, making sure to touch on the following key aspects:

- Was the frame captured by the Chief Engineer correctly decoded?
- Were you able to concoct a plan that requires fewer messages?
- Could you decode the prank memo? Was it classy?

11.7 Mission 7: To TCP or not to TCP



Years of consistent success have consolidated the use of IPv4 for internet-working both within Mars and with other planets. During this time, the engineering teams of the colony have discovered the limitations of bare IPv4 and the need to use at least one of the main transport protocols when communicating with Earth's Internet.

Implementing this fourth layer and adding it to the existing network stacks of each node in the colony is a daunting task. Thus, in a first phase only one of the two main transport protocols will be implemented. The colony is asking for your help to quantitatively compare these two protocols and help decide which one to pick.

⚠ Warning! This mission is highly critical. Take your time, request as much help as needed, and keep your spirits up. Previous missions can be regarded as training for this one — it is strongly recommended you carefully finish them before attempting this one.

Report details

The assembly has requested that you carefully consider the following points in your report:

1. To make your report standalone, include the colony's network topology using the CIDR format when applicable, and specify reasonable MTU values (you may reuse the ones you used in previous missions). Complete this description by specifying the bandwidth (link speed, in bytes per second – let us assume it is constant).
2. One of the two candidate transport protocols uses a parameter called MSS. Indicate which one, and specify the largest value that would allow communication between any two machines of the colony without requiring fragmentation. Use that value in the remainder of your report.
3. Pick a node N_1 from a base directly connected to the North Bus. Pick another node N_2 from a base directly connected to the South Bus. For each of the two main transport protocols, calculate the total time elapsed from when N_1 starts transmission to when N_2 receives the first byte of payload data. To do so, assume:
 - If the protocol is connection-oriented, data are only transferred once the connection is established.
 - All nodes and routers have infinite computing power, i.e., no delay is introduced by encapsulating/decapsulating data.
 - Transmission speeds are still finite, and as you defined earlier in your report.
 - Routing tables are optimal.
 - All caches begin empty for each protocol, and entries remain valid once they are set.

You are encouraged to include swimlane diagrams to support your answer.

4. Which of the transport protocols include mechanisms for congestion control? What conditions would be required to allow continuous transmission from N_1 to N_2 without waits due to congestion control?
5. Indicate three use cases, i.e., user applications: the first one suitable only for one of the transport protocols, the second one suitable for the other transport protocol, and the third one suitable for both.

Feedback details

Use a similar feedback process as in the previous missions, making sure to touch on the following key aspects:

- Is the chosen MSS consistent with the specified MTU values?
- Is the transmission time of each packet correctly calculated?
- Do you see any problem with routing, fragmentation or ARP?
- Are all elements of congestion control properly considered?
- Are the proposed use cases well chosen?

11.8 Mission 8: You talking to me?



Your colony has been successfully using (according to Saturn, also *abusing*) the TCP/IP stack to connect to Earth's Internet. However, until now, in a sense you didn't exist because you didn't have a *name*. That is about to change, because the colony is installing its first Domain Name System (DNS) server, which will be authoritative for the new `.mars` top-level domain (TLD). Your help is needed to help configure new services offered to the whole Solar System.

Report details

1. Summarize the network topology of your colony, and include the IP address ranges and MTU values assigned to each LAN using CIDR notation. This will make your report standalone and help reviewers provide feedback (you may reuse or update the configuration of previous sessions).
 2. Clearly indicate what machine will be hosting your DNS server, and its IP address.
 3. Specify 5 services $S_1, S_2, \dots, S_5$ (different from DNS) that will be hosted in your colony. One of them will be hosted on the same machine as the DNS server. For each service, provide:
 - A brief explanation of the service and the application-layer protocol employed.
 - The DNS entry or entries related to this service to be included in the new DNS server.
 - An example query (e.g., `service.mars`) for which those entries will be returned, who will send that query, and when.
 4. The machine hosting the DNS server also hosts another service. How can it distinguish between the two types of traffic?
 5. Someone using node N_1 in your colony will implement a script to detect when Internet services go offline. To do so, they will employ periodic ICMP messages to *check* they are online. Explain how exactly this will work and, using swimlane diagrams, describe all traffic within the colony derived from a single *check* for each of the following scenarios:
 - (a) N_1 checks service `cv.uab.cat` using iterative resolution over UDP starting on root server M.
 - (b) N_1 checks service S_1 (indicate its name) using iterative resolution over TCP starting on root server M.
 - (c) N_1 checks service S_1 using recursive resolution over UDP with the unfiltered DNS4EU server.
- In the diagrams, for each exchanged message, indicate:
- The (top-layer) message type and, if applicable, its subtype.
 - All involved addresses.
 - All DNS-related information contained in the message.

Feedback details

Consider similar requirements and process as in the previous missions, making sure to touch on the following key aspects:

- Is ICMP being used in a logical, effective way?
- Are iterative and recursive DNS resolution properly handled?
- Is the TCP connection lifecycle correctly represented?

11.9 Mission 9: Interplanetary Hallway



Thanks in part to your help configuring the colony's network and services, your colony is thriving with cultural and scientific activity. The main hallway of your colony's interplanetary spaceport sees thousands of visitors from different planets every day. These visitors demand quick and simple Internet connectivity from this hallway, and the colony assembly has appointed you to help with the configuration.

Report details

The assembly has asked you to configure a DHCP server for the passenger devices. The Autonomous System number 7847 is gracefully lending you the highest 1024 addresses of their block, which will be the ones provided by this service.

Every 2 hours, a ship with up to 300 people (each with a single, Internet-hungry device) arrives at the spaceport. Some depart immediately while others stay longer in the spaceport, but at most 1000 devices need to be connected at the same time. Right before the spaceport begins operation in the morning, all addresses managed by the DHCP server must be available for leasing, because the first 5 ships are completely full.

1. Find out who is generously lending you the 1024 public IP addresses, and what their full block is.
2. Provide an updated routing table for router R_0 . This is the router directly connected to the colony's HugeAntenna™, the North and South Buses, and now to the new LAN dedicated to passengers. You may assume this new LAN is connected to a newly added interface.
3. What is the longest Lease Time you can configure that is compatible with the morning rush? Justify your answer.
4. Using swimlane diagrams, provide a concrete scenario in which the maximum of 1000 devices is simultaneously connected at some point during the rush hour. Indicate how many messages of each DHCP subtype are involved.
5. If a passenger hid in the spaceport terminal during the night, what mechanism would you use to ensure their assigned IP address is available for leasing the next morning?

Feedback details

Consider similar requirements and process as in the previous missions, making sure to touch on the following key aspects:

- Is the routing table correct, and as simple as possible?
- Did you find a longer Lease Time that is compatible with the requirements?
- Do you see any issue with the 1000 concurrent device scenario description?

Index of Concepts

- 3-way handshake, [49](#)
- A, [57](#)
- AAAA, [57](#)
- ACK, [45](#), [48–50](#)
- acknowledgement
 - number, [49](#), [52](#)
- address, [13](#), [14](#), [18](#), [20](#)
- address block, [29](#)
- amplitude, [11](#)
- analog, [25](#)
- antenna, [10](#), [11](#)
- ARP, [28–30](#), [32](#), [37](#), [42](#)
- ARPANET, [25](#)
- AS, [60](#)
- ASCII, [8](#)
- authentication, [28](#)
- authoritative, [57](#)
- ballpark, [11](#)
- bandwidth, [30](#)
- BGP, [61](#)
- big endian, [8](#), [22](#)
- binary, [8](#)
- binary mode, [18](#)
- bind, [46](#), [47](#)
- bitdepth, [8](#)
- bitwise, [9](#)
- body, [17](#)
- boolean logic, [9](#)
- BR, [61](#)
- broadcast, [29](#), [30](#), [35](#), [41](#), [43](#)
- buffer, [50](#)
- bus, [12](#), [13](#), [16](#), [17](#)
- byte boundary, [17](#)
- cache, [43](#), [58](#)
- carrier, [11](#)
- channel, [17](#)
- checksum, [38](#)
- CIDR, [34](#), [39](#)
- client, [13](#), [30](#), [46](#)
- clock, [10](#)
- CNAME, [57](#)
- collision, [12](#)
- complete graph, [12](#)
- concatenation, [9](#)
- connection, [25](#), [45](#)
- copper wire, [10](#)
- cryptography, [28](#)
- data, [8](#)
- data line, [10](#), [17](#)
- data link, [16](#)
- datagram, [20](#), [28](#), [32](#), [37](#), [38](#), [41](#), [46](#), [52](#), [56](#)
- decapsulate, [37](#)
- decode, [18](#)
- default routing table, [38](#)
- device, [12](#)
- DF, [41](#)
- DHCP, [58](#)
- digital, [25](#)
- dipole, [11](#)
- direct delivery, [38](#)
- DNS, [27](#), [56](#)
- DNS record, [57](#)
- domain name, [56](#)
- echo, [30](#)
- encapsulation, [20](#), [21](#), [28](#), [29](#)
- encoding, [8](#)
- end-to-end, [14](#)
- error detection, [17](#)
- escape sequence, [16](#)
- Ethernet, [28](#), [30](#)
- EtherType, [30](#)
- exponential, [12](#)
- exterior gateway protocol, [61](#)
- field, [10](#)
- field (packet), [17](#)
- field (physics), [11](#)
- FIN, [48](#), [53](#)
- fixed-point, [8](#)
- flag, [17](#)
- floating-point, [8](#)
- flow control, [45](#)
- fragmentation, [32](#), [41](#), [46](#)
- frame, [20](#), [29](#), [30](#)
- frequency, [11](#)
- full duplex, [50](#)
- gratuitous, [43](#)
- header, [17](#), [20](#), [21](#), [36](#), [37](#), [41](#), [47](#)
- hexadecimal, [9](#), [14](#), [18](#)
- hop, [37](#), [60](#), [61](#)
- host, [45](#)
- HTTP, [46](#)
- HTTPs, [46](#)
- hub, [13](#)
- IANA, [27](#)
- ICANN, [27](#)
- ICMP, [33](#), [44](#)
- IETF, [26](#), [27](#)
- indirect delivery, [38](#), [42–44](#)
- information, [8](#)
- integer, [8](#)
- interface, [37](#)
- interior gateway protocol, [61](#)
- internal router, [61](#)
- Internet, [24](#), [25](#), [30](#)
- IP, [32](#), [36](#), [42](#), [44](#), [46](#), [56](#)
- IP address, [34](#)
- IP address block, [34](#)
- IP Options, [38](#)
- IPv4, [28–30](#), [32](#), [33](#), [39](#), [46](#), [52](#)
- IPv6, [28](#), [29](#), [32](#), [33](#), [46](#), [52](#)
- ISP, [57](#)
- iterative, [57](#)
- IXP, [60](#)
- LAN, [13](#), [14](#), [25](#), [28](#), [30](#), [32](#), [39](#)
- layer, [20](#), [21](#), [26](#), [50](#)
- lease, [59](#)
- link local, [33](#)
- little endian, [8](#)
- local broadcast, [59](#)
- MAC, [28](#), [32](#), [37](#), [42](#)

medium, 10
message, 8, 18
metadata, 17
MF, 41, 42
modulation, 11
monochrome, 10
MSS, 52
MTU, 16, 28, 29, 32, 41, 52
multicast, 29
multimodal, 10
multiplex, 10, 17
MX, 57

netmask, 33, 34, 39
network, 13, 18, 25, 35
network card, 12
network interface, 14
network stack, 23
NP, 60
NS, 57

Offset, 41, 42
offset, 19
optical fiber, 10
OS, 13
overhead, 16, 18, 20, 21, 25, 41, 45, 52

packet, 16, 18, 21–23, 25, 36
packet format, 17, 18, 20, 26
packet structure, 17
packet switching, 16
parallel, 10
parsing, 17
path, 18, 61
payload, 17, 20–22, 41, 44, 45
PDU, 20–22, 26, 28, 32
peer-to-peer, 30
phase, 11
physical layer, 23
piggyback, 51
ping, 44
point-to-point, 12

poison, 43
pool, 59
port, 13, 44–46, 49
power, 30
preamble, 16
private IP, 39
privative, 27
process, 13, 14, 45, 47, 54
protocol, 24–26, 28
PSH, 50
public, 38, 39
public IP, 33

quad decimal, 33–35, 41
quadratic, 12
query, 57

recursive, 57, 58
refraction, 10
reliability, 45, 49, 50
renew, 59
reply, 57
request, 57
reserved IP address, 33
retire (standard), 27
retransmission, 45, 51
RFC, 26, 27
RIP, 61
RIR, 27
root DNS, 56
router, 14, 18, 35, 37, 41, 42, 44
routing, 13, 14, 32
routing errors, 39
routing table, 37, 38, 44, 60, 61
RST, 48, 54
RTT, 51

scan, 43
segment, 20, 47, 50
sequence number, 49
sequencing, 20
serial, 10
server, 13, 30, 46
service, 13

sign, 8
single-mode optical fiber, 10
sniff, 18
socket, 47–49
sshd, 46
stack, 20–22, 25
star topology, 13
stateful, 48
stream, 18, 25, 32, 50
subnet, 39, 40
subnetting, 40
suit, 24
supernet, 40
swimlane diagram, 15
switch, 13, 14, 30, 43
symbol, 8
SYN, 48, 49

tail, 17
TCP, 25, 32, 33, 45–47, 56
TCP/IP, 25, 28
text mode, 18
throughput, 17
Token Ring, 12
top-level domain, 56, 57
topology, 12
trailer, 17
transmission, 8
transmission window, 52
transport, 45
TTL, 38, 39, 44
twisted pair, 10

UDP, 25, 33, 45, 46, 56
unicast, 30
unsigned, 22
user datagram, 45–47

virtual, 20, 21

WAN, 13, 14
Wi-Fi, 28

zone, 57